# A Web-Based Unified School Management System for Academic and Administrative Operations with Role-Based Access

---

## Final Year Project Report

by


| | |
|---|---|
| **Saiha Atiq** | **BCS22007** |
| **Malaika Ashraf** | **BCS22015** |
| **Khadija Mustafa** | **BCS22043** |
| **Zainab Khawaja** | **BCS22049** |

Project Advisor:

Sir Salman Naseer

---

Department of Information Technology

University of the Punjab, Gujranwala Campus, Gujranwala, Pakistan


(2026)

A Web-Based Unified School Management System for Academic and Administrative Operations with Role-Based Access

## Executive Summary

*This project presents a School Management System (SMS) designed to streamline academic and administrative processes through a role-based digital platform. The Project Proposal defines the problem scope, objectives, stakeholders, and feasibility, focusing on improving efficiency, transparency, and communication. The Software Requirements Specification (SRS) details functional and non-functional requirements, role-based access for Admin, Teacher, Student, and Parent, and core features including attendance, marks, fees, complaints, messaging, notices, portfolios, and audit logging, while addressing usability, performance, accessibility, and security.*

*The Software Design Specification (SDS) translates these requirements into a structured design, outlining system architecture, module interactions, data flow, and class relationships while keeping the UI unchanged. A functional frontend prototype using mocked data demonstrates realistic user flows, role-based permissions, validations, notifications, session handling, and audit logs without a backend. Together, the proposal, SRS, SDS, and prototype provide a complete and consistent view of the system from concept to working demonstration.*

# FYDP Overview

**FYDP Title: <u>A Web-Based Unified School Management System for Academic and Administrative Operations with Role-Based Access</u>**

| Sr. No | Roll Numbers | Name | Signatures |
|---|---|---|---|
| 1. | BCS22007 | Saiha Atiq | |
| 2. | BCS22015 | Malaika Ashraf | |
| 3. | BCS22043 | Khadija Mustafa | |
| 4. | BCS22049 | Zainab Khawaja | |

*Table 1 Project Proposal Summary*

| **FYDP Goals** |
|---|
| • Develop a **centralized platform** to efficiently manage **school activities** including **classes, subjects, fees, attendance, and exam marks**. |
| • Provide **students, teachers, and parents** with **real-time access** to **attendance, marks, and risk analysis** for improved **academic monitoring**. |
| • Implement a **communication hub** and **digital noticeboard** to centralize **announcements, messages, and feedback** between **teachers and parents**. |
| • Create **automated student portfolios** and a **complaint/behavior management system** to track **achievements** and support **student development**. |
| • Design and implement a **scalable, modular web-based system** that **integrates students, teachers, parents, and administrators** into a unified platform. |
| **FYDP Objectives** |
| • Digitize and streamline **school administrative tasks** to improve **accuracy and efficiency**. |
| • Enable **real-time tracking** of **student performance, attendance, and risk factors**. |
| • Facilitate **effective communication** between **teachers, students, and parents**. |
| • Generate **digital student portfolios** and manage **complaints and behavior** in a systematic manner. |

| |
|---|
| **FYDP Success Criteria** |
| • All school data including students, teachers, classes, subjects, fees, attendance, and marks can be managed digitally with improved accuracy and reduced errors. |
| • Students, teachers, and parents can access real-time information on performance, attendance, and notifications. |
| • Communication between teachers, students, and parents is centralized, timely, and reliable. |
| • Student portfolios are automatically generated, and complaints and behavior records are tracked and resolved efficiently. |
| **Assumptions:** |
| • Data Access: Sample school data, including student attendance, marks, and basic records, will be available or generated for testing and demonstration purposes. |
| • Pilot Access: A single school or class can be used to conduct a pilot study and gather feedback on usability and functionality. |
| • User Engagement: Teachers, students, and parents will actively participate in testing, providing feedback to refine the system. |
| **Risks & Obstacles** |
| • Limited Technical Expertise: Some users may have minimal experience with web-based systems, potentially affecting adoption. |
| • Data Privacy and Trust: Parents and school staff may have concerns regarding the security and confidentiality of student data. |
| • Integration Challenges: Incorporating the system alongside existing school records or processes may face compatibility issues. |
| • Infrastructure Limitations: Slow internet connections or older devices in some schools could hinder system performance and responsiveness. |
| **Organization Address:** University of the Punjab, Gujranwala Campus, Pakistan |
| **Target End Users: Students, Teachers, Parents, Administrators** |
| **Suggested Project Supervisor: Sir Salman Naseer** |
| **Approved By:** |
| **Date: February 02, 2026** |

## Table of Contents

*A Web-Based Unified School Management System for Academic and Administrative Operations with Role-Based Access* ..... 3

**Executive Summary** ..... 3

*FYDP Overview* ..... 4

*List of Tables* ..... 9

*1.1 Introduction* ..... 13

*1.2 Background* ..... 13

*1.3 Problem Statement* ..... 13

*1.4 Stakeholders & Interests* ..... 14

*1.5 Objectives:* ..... 15

*1.6 Scope:* ..... 15

*1.7 Assumptions* ..... 16

*1.8 Risks* ..... 17

*1.9 Success Criteria* ..... 18

*1.10 Tools, Libraries & Technologies* ..... 19

*1.11 Work Division* ..... 20

*1.12 Conclusion* ..... 21

*Literature Review* ..... 22

*2.1 Literature Survey* ..... 23

*2.2 Related Work:* ..... 23

*2.3 Gap Analysis:* ..... 24

*3.1 Requirements Analysis* ..... 27

*3.2 User classes and characteristics* ..... 27

*3.3 Requirement Identifying Technique* ..... 27

*3.4 Functional Requirements* ..... 28

User Authentication and Authorization ..... 28

Attendance Management ..... 30

Marks and Student Performance ..... 31

Student Portfolio Generation ..... 34

Noticeboard & Notifications ..... 35

Parent–Teacher Messaging ..... 36
Complaint and Behavior Management ..... 37
Fee Reminder ..... 38
Audit Logs & System Monitoring ..... 39
***3.5 Non-Functional Requirements ..... 39***
1. Reliability ..... 39
**Usability** ..... 40
**2. Performance ..... 40**
**3. Security ..... 41**
***3.6 External Interface Requirements ..... 42***
1. User Interfaces Requirements ..... 42
2. Software interfaces ..... 43
3. Hardware interfaces ..... 44
4. Communications interfaces ..... 45
***3.7 Use case Analysis ..... 46***
**1. UC1 – User Login & Role-Based Access ..... 46**
**2. UC2 – Account Management (Admin) ..... 47**
**3. UC3 – Parent Account Verification ..... 47**
**4. UC4 – Dashboard View ..... 48**
**5. UC5 – View Attendance (Student) ..... 49**
**6. UC6 – Mark Attendance (Teacher) ..... 49**
**7. UC7 – Enter Marks (Teacher) ..... 50**
**8. UC8 – Upload Achievements (Teacher) ..... 51**
**9. UC9 – Record Behavior Remarks (Teacher) ..... 51**
**10. UC10 – View Student Portfolio (Student/Parent) ..... 52**
**11. UC11 – Student Risk Monitoring (System) ..... 52**
**12. UC12 – Submit Complaint (Student/Parent) ..... 53**
**13. UC13 – Resolve Complaint (Teacher/Admin) ..... 54**
**14. UC14 – Notify Users (System) ..... 54**
**15. UC15 – Post Notices (Teacher/Admin) ..... 55**
**16. UC16 – Parent–Teacher Messaging ..... 55**
**17. UC17 – View Fee Status (Student/Parent) ..... 56**
**18. UC18 – Manage Fees (Admin) ..... 57**
**19. UC19 – Manage Students (Admin) ..... 57**
**20. UC20 – Manage Teachers & Subjects (Admin) ..... 58**

21. UC21 – Review Attendance & Marks (Admin) ..... 58
22. UC22 – Compile Student Portfolio (System) ..... 59
23. UC23 – Password Reset / Change ..... 60
24. UC24 – Audit Logging (System) ..... 60
25. UC25 – Generate Reports (Admin) ..... 61
26. UC26 – Logout (User) ..... 61
Use Case Diagram ..... 63
*3.8 Storyboards* ..... *64*
*3.9 Summary* ..... *71*
*4.1 System Design* ..... *73*
*4.2 Design Considerations* ..... *73*
4.3 Requirements Traceability Matrix ..... 74
4.4 Design Models ..... 77
Design Class Diagram (DCD) ..... 77
2. Interaction Diagram (sequence) ..... 78
3. State Transition Diagram ..... 80
*4.5 Architectural Design* ..... *84*
1. UML Component diagram ..... 87
*4.6 Data Design* ..... *88*
1. Data Dictionary ..... 89
*4.7 User Interface Design* ..... *108*
1. Screen Images ..... 109
2. Screen Objects and Actions ..... 111
*4.8 Design Decisions* ..... *112*
*4.9 Summary* ..... *116*
*References* ..... *118*
1. Article in a Journal ..... 118
2. Articles from Conference Proceedings (published) ..... 118
3. World Wide Web ..... 118
4. ChatGPT ..... 119

# List of Tables

Table 1 Project Proposal Summary....4
Table 2 Project Success Criteria....18
Table 3 Tools Technologies and Libraries....19
Table 4 Project Team Members Work Division....20
Table 5 User classes....27
Table 6 Defines the system requirements for secure user authentication and redirection to role-specific dashboards....28
Table 7 Specifies administrative capabilities for creating, updating, deactivating, and managing user accounts and roles....29
Table 8 Describes the functionality that allows teachers to record daily student attendance for assigned classes....30
Table 9 Defines the requirements for teachers to enter and manage student marks for tests and examinations....31
Table 10 Specifies the process for uploading and managing student achievements and certificates as part of the digital portfolio....31
Table 11 Describes how teachers can record and manage student behaviour remarks for academic and disciplinary tracking....32
Table 12 Defines automated analysis of attendance, marks, and behaviour to generate student risk indicators....33
Table 13 Specifies the system's ability to compile attendance, marks, behaviour, and achievements into a unified student portfolio....34
Table 14 Describes the functionality for authorized staff to publish notices to students and parents....35
Table 15 Defines the notification mechanism for informing users about notices and important system updates....35
Table 16 Specifies secure communication requirements between parents and teachers within the system....36
Table 17 Defines the process through which students and parents can submit formal complaints....37
Table 18 Describes how teachers and administrators manage and resolve submitted complaints....37
Table 19 Specifies the system requirements for displaying student fee status to authorized users....38
Table 20 Defines the system's logging requirements for tracking critical actions and ensuring accountability....39
Table 21 UC1 – User Login & Role-Based Access – Authenticate users and route to role-specific dashboards....46
Table 22 UC2 – Account Management (Admin) – Enable administrators to create, update, deactivate, or delete user accounts and assign system roles....47
Table 23 UC3 – Parent Account Verification – Verify parent accounts and activate secure login credentials....47
Table 24 UC4 – Dashboard View – Display a role-based dashboard with relevant widgets, shortcuts, and system information....48

Table 25 UC5 – View Attendance (Student) – Allow students to view subject-wise attendance records and trends. ..... 49
Table 26 UC6 – Mark Attendance (Teacher) – Allow teachers to record daily attendance for assigned classes. ..... 49
Table 27 UC7 – Enter Marks (Teacher) – Enable teachers to enter and manage student marks for exams and assessments. ..... 50
Table 28 UC8 – Upload Achievements (Teacher) – Allow teachers to upload student certificates and extracurricular achievements. ..... 51
Table 29 UC9 – Record Behavior Remarks (Teacher) – Record positive or negative behavior remarks for students. ..... 51
Table 30 UC10 – View Student Portfolio (Student/Parent) – Provide students and parents access to a consolidated student portfolio. ..... 52
Table 31 UC11– Student Risk Monitoring (System) – Automatically analyze attendance, marks, and behavior to determine student risk levels. ..... 52
Table 32 UC12 – Submit Complaint (Student/Parent) – Allow students and parents to submit formal complaints within the system. ..... 53
Table 33 UC13 – Resolve Complaint (Teacher/Admin) – Enable staff to review, manage, and resolve submitted complaints. ..... 54
Table 34 UC14 – Notify Users (System) – Send automated notifications for system events and updates. ..... 54
Table 35 UC15 – Post Notices (Teacher/Admin) – Allow authorized staff to publish notices on the digital noticeboard. ..... 55
Table 36 UC16 – Parent–Teacher Messaging – Enable secure private communication between parents and teachers. ..... 55
Table 37 UC17 – View Fee Status (Student/Parent) – Allow students and parents to view fee status and download challans. ..... 56
Table 38 UC18 – Manage Fees (Admin) – Enable administrators to manage and update student fee records ..... 57
Table 39 UC19 – Manage Students (Admin) – Allow administrators to add, update, and manage student profiles. ..... 57
Table 40 UC20 – Manage Teachers & Subjects (Admin) – Enable administrators to manage teacher profiles and subject/class assignments. ..... 58
Table 41 UC21 – Review Attendance & Marks (Admin) – Admin reviews and corrects student attendance and marks. ..... 58
Table 42 UC22 – Compile Student Portfolio (System) – System compiles attendance, marks, behavior, and achievements into a portfolio. ..... 59
Table 43 UC23 –Password Reset / Change – Users can securely reset or change their password. ..... 60
Table 44 UC24 –Audit Logging (System) – System maintains immutable logs of all critical actions. ..... 60
Table 45 UC25 –Generate Reports (Admin) – Admin generates academic and administrative reports in PDF/CSV format. ..... 61
Table 46 UC26 – Logout (User) – Users can securely log out of the system. ..... 61

## Tables of Figures

Figure 1 Use Case Diagram ..... 63
Figure 2 Class Diagram ..... 77
Figure 3 Sequence Diagram 1- User Flow Login ..... 78
Figure 4 Sequence Diagram 2- Teacher Academic Management Flow ..... 78
Figure 5 Sequence Diagram 3- Student Portfolio Generation Flow ..... 79
Figure 6 State Diagram 1- Attendance Record ..... 80
Figure 7 State Diagram 2- Complain Mangement ..... 81
Figure 8 State Diagram 3- Student Portfolio ..... 82
Figure 9 State Diagram 4- fee Payment ..... 83
Figure 10 State Diagram 5- User account ..... 84
Figure 11 System Architecture Diagram ..... 85
Figure 12 UML Component Diagram ..... 87
Figure 13 Screen Image 1- Login Screen ..... 109
Figure 14 Screen Image 2- Student Portfolio ..... 110
Figure 15 Screen Image 3- Attendance Management ..... 110
Figure 16 Screen Image 4- Complain Feedback ..... 111
Figure 17 Screen Image 5- Student Portfolio Generation ..... 111

# Chapter No 1
# Project Proposal

## 1.1 Introduction

This chapter provides an overview of the Final Year Design Project (FYDP) proposal for a **Web-Based Unified School Management System (SMS)**. It explains the **background and motivation** for a centralized digital platform to improve **administrative and academic processes** in Pakistan. The chapter defines the **problem statement**, **key stakeholders**, and **objectives and scope** of the system.

It presents the FYDP overview, outlining the **project goals, success criteria, assumptions, potential risks, tools and technologies to be used, and team work distribution.** Overall, it establishes a clear foundation for defining **system requirements, design approach, feasibility considerations, and implementation expectations.**

## 1.2 Background

Schools in Pakistan frequently rely on fragmented administrative channels such as paper records, spreadsheets, and informal messaging platforms (e.g., WhatsApp) to manage attendance, grades, student achievements, and parent communication. This disjointed approach often results in inconsistent record-keeping, delayed information flow, privacy concerns, and limited transparency in student progress tracking. Teachers and administrators face challenges in maintaining accurate records and ensuring effective communication, while parents lack a centralized view of their child's academic and behavioral history. A web-based School Management System (SMS) addresses these challenges by providing a centralized, role-based platform for students, parents, teachers, and administrators. The system enables streamlined management of **academic records, behavior tracking, communication, and notifications, reducing errors and administrative overhead while enhancing transparency, coordination, and accountability.** By consolidating essential school operations into a maintainable and practical digital solution, the SMS offers immediate institutional value and supports informed decision-making across all stakeholders.

## 1.3 Problem Statement

Schools in Pakistan still rely on paper-based records, informal communication channels, and disconnected processes for managing academic and administrative tasks. As a result, attendance, results, fee tracking, and communication often become inaccurate, delayed, or difficult to monitor. Most schools also lack:

1. A **centralized student portfolio** containing grades, attendance, achievements, and certificates.
2. A secure, auditable **parent–teacher communication** platform instead of WhatsApp groups.
3. A unified **digital noticeboard** for official announcements.
4. A structured **complaint/behavior management** workflow with proper tracking.

A centralized, secure, and scalable School Management System is needed to streamline school operations, enhance communication, ensure accurate record-keeping, and improve transparency and efficiency for students, teachers, parents, and administrators.

## 1.4 Stakeholders & Interests

| Stakeholder Type | Interest in the Solution | Potential Market size |
|---|---|---|
| **Students** | Access their attendance, results, fee status, digital portfolio, and complaint status through a secure dashboard; submit complaints; ensure privacy of academic and behavioral records. | 25–30 million school students in Pakistan |
| **Teachers / Tutors** | Mark student attendance, enter exam marks, upload achievements/certificates for portfolios, post announcements/homework, provide remarks, and respond to parent messages and student complaints.. | 1.5–2 million school teachers nationwide |
| **Parents** | View their child's attendance, results, portfolio, and fee status; communicate privately with teachers; receive official school notices and updates. | Approx. 40–50 million parents/guardians |
| **School Management / Administration / PTA** | Oversee overall school performance by managing classes, subjects, students, teachers, teacher–subject mappings, fees, and exam records; monitor reports, notices, complaint summaries, and student portfolios; post school-wide announcements; maintain system data; and ensure transparency while improving academic and administrative oversight. | Approx. 300,000 administrative staff |

## 1.5 Objectives:

Following is a list of the main objectives of the system to be designed:

- Build a **secure, role-based School Management System** for Students, Parents, Teachers, and Admin with proper authentication and data privacy.
- Develop a **Student Portfolio Generator** compiling academic performance, attendance, achievements, and activities.
- Create a **Parent–Teacher Communication Hub** for private messaging, announcements, and dashboard notifications.
- Implement a **Digital Noticeboard** for school-wide announcements with optional email/SMS alerts.
- Design a **Behaviour / Complaint Management module** for submitting, tracking, and resolving student complaints.
- Analyze and address **risks related to students' attendance and academic performance.**
- Deploy a **functional prototype with sample data, testing, and documentation**.

## 1.6 Scope:

**Inclusions:** The School Management System provides role-based dashboards for Students, Parents, Teachers, and Admin, focusing on practical and essential school operations. It features a Student Portfolio Generator that aggregates grades, attendance, achievements, and activities, a Parent–Teacher Communication Hub for announcements and private messaging, and a Digital Noticeboard for school-wide notifications such as exams, holidays, PTA meetings, and fee reminders. The Behaviour and Complaint Management module allows students to submit concerns, teachers to add remarks, and admins to resolve and track cases. Teachers handle attendance and marks entry, while students and parents can access academic progress and fee status, ensuring transparency, effective communication, and streamlined school management.

**Exclusions/Limitations:** The system does not include automated timetable generation or schedule optimization algorithms, nor AI-based predictions, analytics, or machine learning modules. Advanced fee accounting or comprehensive financial management features are excluded, with only fee status updates and reminders provided. Large-scale third-party integrations, such as ERP systems or full SMS gateways, are not implemented, although optional email or SMS notifications may be supported if feasible. Additionally, the system will not have dedicated native mobile applications, relying instead on a fully responsive web interface accessible via browsers on desktops and mobile devices.

## 1.7 Assumptions

The system design and implementation are based on the following assumptions, which directly influence architectural and functional decisions:

**1. Availability of Sample Data**
It is assumed that sample data for students, teachers, attendance records, examination results, and fee details will be available or can be artificially generated. This assumption allows the system to be properly tested and demonstrated during development and FYDP evaluation without requiring access to real institutional data, ensuring privacy and feasibility.

**2. Single-School Deployment**
The system is initially designed for deployment in a single school or campus environment. This assumption simplifies the architecture by avoiding complex multi-tenant requirements such as cross-school data isolation, large-scale user management, and distributed databases, making the system suitable for academic and prototype-level implementation.

**3. Stable Internet Access**
It is assumed that users will have at least basic and stable internet connectivity. Since the system is web-based, continuous internet access is required for real-time data access, updates, and role-based interactions between administrators, teachers, students, and parents.

**4. Role Awareness**
The system assumes that users (Admins, Teachers, Students, and Parents) have a basic understanding of their roles and responsibilities within the system. This allows the interface and workflows to remain simple and intuitive without extensive training modules or onboarding mechanisms.

**5. Web-Based Usage Only**
The system is designed to be accessed exclusively through web browsers. Native mobile applications are not included in the scope of this project. This assumption reduces development complexity and ensures platform independence, allowing access from desktops, laptops, and mobile browsers.

**6. Prototype-Oriented Development**
Some advanced features such as notifications, file uploads, and analytics may be simulated or partially implemented for demonstration purposes. This assumption aligns with the FYDP scope, time constraints, and academic evaluation requirements, while still showcasing the system's intended functionality.

## 1.8 Risks

The following risks may arise during the development and deployment of the FYDP, along with strategies to minimize their impact:

**1. Limited Technical Skills of End Users**

Teachers or parents may face difficulties adapting to a digital system due to limited technical familiarity, which can affect system adoption and effective usage.

**Mitigation:** Design intuitive and user-friendly dashboards with clear navigation, validation messages, and minimal-click workflows to ensure ease of use for all user groups.

**2. Data Privacy and Security Concerns**

Stakeholders may be concerned about unauthorized access, data leakage, or misuse of sensitive student and academic records.

**Mitigation:** Implement strong security measures including role-based access control, secure authentication, session management, and audit logging as defined in the Software Requirements Specification (SRS).

**3. Internet and Infrastructure Limitations**

Slow internet connections or the use of older devices may negatively impact system responsiveness and overall usability.

**Mitigation:** Optimize frontend performance by minimizing resource usage, applying lazy loading techniques, and ensuring that system pages load within the defined Non-Functional Requirements (NFRs).

**4. Time Constraints of FYDP**

Limited development time may restrict the implementation of all planned features and affect overall system completeness.

**Mitigation:** Prioritize the development of core modules such as authentication, attendance management, marks management, student portfolios, and communication features to deliver a stable and functional prototype.

**5. Requirement Changes During Development**

Continuous stakeholder feedback may result in changing or evolving requirements during the development lifecycle.

**Mitigation:** Adopt a modular system design and ensure that the Software Requirements Specification (SRS) and Software Design Specification (SDS) are regularly updated to reflect approved changes.

## 1.9 Success Criteria

The system will be accepted as successful if all the following features and conditions are met:

*Table 2 Project Success Criteria*

| **Category** | **Acceptance Criteria** |
|---|---|
| User Authentication | Secure login with role-based dashboard redirection |
| Role-Based Access Control | Admin, Teacher, Student, and Parent access limited to authorized modules |
| Core Academic Functions | Attendance marking, marks entry, and performance indicators |
| Administrative Functions | Fee status management, notices, and user management |
| Student Portfolio | Automatic compilation of attendance, marks, achievements, and behavior |
| Communication System | Secure parent–teacher messaging and dashboard notifications |
| Complaint & Behavior Management | Complaint submission, tracking, and resolution workflow |
| Data Management | Validated CRUD operations with error handling |
| Reporting and Visualization | Generate reports and visualize attendance, marks, and student performance |
| Speed and Responsiveness | System operations meet defined response time requirements |
| Scalability | System supports future increase in students, teachers, and classes |
| Data Privacy | Data protection, session expiry, and restricted access |
| Access Control | Role-based authorization enforced for all modules |

# 1.10 Tools, Libraries & Technologies

*Table 3 Tools Technologies and Libraries*

| | **Tools** | **Version** | **Rationale** |
|---|---|---|---|
| **Tools, Libraries, And Technologies** | Visual Studio 2022 / VS Code | Latest | Version Control: IDE for debugging, Development, and Code management |
| | GitHub | Latest | Version Control Platform for source Code management And team Collaboration. |
| | Figma | Latest | Used for designing UI/UX wireframes And application Layout. |
| | **Libraries** | **Version** | **Rationale** |
| | React.js | V18+ | Frontend library for Building dynamic And component-Based user Interfaces. |
| | ASP.NET Core | 7.0+ | Backend frame-work for develop-ing secure and scalabe RESTful APIs. |
| | Entity Framework Core | 7.0+ | ORM for database access and schema management |
| | Bootstrap / Tailwind CSS | Latest | CSS frameworks for responsive and mobile friendly UI design. |

| | | | |
|---|---|---|---|
| | SignalR | Latest | Enables real time communication and notifications |
| | **Technology** | **Version** | **Rationale** |
| | SQL Server | 2019+ | Relational database for storing and managing system Data. |
| | .NET Runtime | 7.0+ | Executes ASP.NET Core applications with high performance |

# 1.11 Work Division

*Table 4 Project Team Members Work Division*

| **Sr. No** | **Roll Number** | **Name** | **Role Assignment & Work Division** |
|---|---|---|---|
| 1. | BCS22007 | Saiha Atiq | **Backend & Core Logic:**Designs ASP.NET Core Web APIs, creates SQL Server database using EF Core, implements role-based authentication, handles core business logic for portfolios, notifications, and risk analysis, oversees integration with frontend. |
| 2. | BCS22015 | Malaika Ashraf | **Frontend & Prototype:**Develops React.js dashboards for all users, ensures responsiveness and interactivity, integrates frontend with ASP.NET Core API, implements mock data for prototype demonstration. |
| 3. | BCS22043 | Khadija Mustafa | **UI/UX Support & Testing Assistance:** Assists in maintaining consistent UI/UX implementation, provides design feedback on prototype usability, supports testing for frontend functionality and accessibility. |
| 4. | BCS22049 | Zainab Khawaja | **QA & Documentation Support:** Assists in system testing and bug reporting, updates SRS/SDS documentation based |

| | | | |
|---|---|---|---|
| | | | on project progress, prepares reports on test results and requirement compliance. |

## 1.12 Conclusion

This chapter outlined the comprehensive overview of the proposed School Management System, including the project background, objectives, scope, tools, and team roles. Key assumptions, potential risks, and success criteria were clearly defined to guide development and evaluation. By establishing these foundations, the chapter ensures a structured approach for implementing a secure, role-based, and efficient system that meets the academic and administrative needs of schools.

# Chapter No 2

# Literature Review

## 2.1 Literature Survey

The literature survey establishes the foundation for the proposed School Management System (SMS) by reviewing and analyzing existing solutions, research, and technologies in the domain of school administration and academic management. This survey identifies the strengths and limitations of current systems and clarifies the specific problems the proposed project intends to address.

Currently, schools in Pakistan rely on fragmented administrative channels, such as paper records, spreadsheets, and informal messaging platforms like WhatsApp, to manage attendance, academic records, fees, and communication. These disconnected systems result in delayed information flow, inconsistent record-keeping, limited transparency, and privacy concerns. Teachers and administrators often face challenges in maintaining accurate records, while parents have minimal real-time access to their child's academic and behavioral performance.

The literature survey highlights the need for a centralized, web-based solution that integrates administrative and academic workflows, supports role-based access, and provides real-time data visibility for students, parents, teachers, and administrators. This foundation informs the design of the proposed system and ensures it addresses gaps present in existing solutions.

## 2.2 Related Work:

Several research studies and existing software solutions have been developed to improve school administration and academic management. These systems mainly focus on digitizing attendance, results, and basic administrative tasks.

- **Ahmed et al. (2022)** examined the role of School Management Systems in Pakistan and found that digital platforms significantly improve administrative efficiency by reducing paperwork and manual errors. However, their study highlights that many systems remain limited to administrative use and do not adequately support parents or students with real-time access.
- **Muslim et al. (2025)** explored digital platforms for parent–teacher communication and emphasized that online systems improve transparency and engagement. Despite this, communication in many schools still relies on informal tools such as WhatsApp, which lack privacy controls, official record-keeping, and audit trails.
- **Urooj et al. (2025)** discussed the importance of maintaining digital student portfolios that combine attendance, grades, achievements, and certificates. Their findings indicate that most existing systems do not provide a unified portfolio, forcing schools to maintain scattered records.
- **Khan et al. (2024)** studied centralized digital noticeboards and concluded that a single official platform for announcements reduces miscommunication. However,

many schools still distribute notices through multiple unstructured channels, leading to missed or delayed information.

- **Ahmed et al. (2023)** proposed structured complaint and behavior management mechanisms within school systems but noted that such features are rarely implemented in commonly used platforms.

From a market perspective, solutions such as EduSuite provide basic school management features like attendance and fee tracking. However, these systems often lack advanced features such as student portfolio generation, structured complaint handling, and dedicated parent modules. Additionally, most market solutions prioritize administrative needs over user-friendly interfaces for parents and students.

Overall, existing research and market solutions partially address school management challenges but fail to provide a fully integrated, role-based, and transparent system that equally supports administrators, teachers, parents, and students.

## 2.3 Gap Analysis:

Despite existing research and available school management solutions, several important gaps remain.

**Competitive Gap (Core Features):**

Most existing systems do not provide integrated modules such as centralized student portfolios, complaint and behavior tracking, digital noticeboards, and secure parent–teacher messaging within a single platform(Urooj, Tariq, & Khan, 2025; Ahmed, Farooq, & Shah, 2023)

**Usability Gap:**

Many systems prioritize administrative functionality but lack simple and user-friendly dashboards for parents and students, limiting accessibility for non-technical users. (Ahmed, Farooq, & Shah, 2023)

**Communication Gap:**

Although research highlights the importance of secure communication, schools still rely on informal platforms like WhatsApp, which lack privacy, security, and auditability (Muslim, Aslam, & Rizwan, 2025)

**Implementation Gap (Record-Keeping & Transparency):**

Existing platforms rarely maintain a complete digital history of announcements, complaints, behavior remarks, and achievements, reducing transparency and accountability. (Ahmed, Farooq, & Shah, 2023; Khan, Ali, & Qazi, 2024)

**Data Integration Gap:**

Student data is often scattered across multiple systems and manual records. Research emphasizes the need for unified data management, which is still missing in most school-level systems. (Urooj, Tariq, & Khan, 2025; Rehman & Ali, 2023)

These gaps justify the need for the proposed School Management System, which aims to provide a centralized, secure, and role-based platform addressing both academic and administrative requirements.

## 2.4 Summary

This chapter presents a comprehensive review of literature and existing systems relevant to school management. It identifies limitations in current solutions, including fragmented data management, lack of centralized portfolios, inadequate communication tools, and insufficient transparency. The gap analysis confirms the need for a **centralized, role-based, secure, and user-friendly platform** that integrates academic and administrative operations. Insights from this chapter guide the proposed system's functional requirements, system architecture, and implementation strategies, ensuring that it addresses real-world challenges faced by schools, teachers, parents, and students.

# Chapter No 3
# Software Requirements Specification

## 3.1 Requirements Analysis

This SRS document presents the analysis and requirements gathered for developing the School Management System (SMS). It summarizes the problem domain, identifies the needs of different user roles, and defines the system's functional and non-functional requirements. The work includes reviewing existing school workflows, studying user expectations, and outlining the features necessary to digitalize core academic and administrative processes. This section establishes the foundation for the system's design and implementation in later stages.

## 3.2 User classes and characteristics

*Table 5 User classes*

| User Class | User Characteristics |
|---|---|
| Student | Accesses attendance, marks, results, and portfolio. Submits complaints and views notifications. Requires simple, intuitive interface. |
| Parent | Views child's progress, attendance, fees, and teacher messages. Receives school notifications. Needs secure and easy-to-use dashboard. |
| Teacher | Marks attendance, enters exam results, uploads certificates and supporting learning materials, posts announcements, and communicates with parents/students. Requires efficient workflow and clear data access. |
| Administrator | Manages students, teachers, classes, subjects, fees, and overall system data. Monitors reports, complaints, and notices. Needs full system access, control, and reporting tools. |

## 3.3 Requirement Identifying Technique

- The requirements for the School Management System (SMS) were identified using a combination of structured and semi-structured techniques to ensure comprehensive coverage of all stakeholder needs. Initial **proposal analysis** provided baseline functional expectations and clarified the project scope. Informal **interviews and discussions** with teachers, students, and administrative staff helped capture real-world workflows, typical challenges, and user-specific requirements.

- To further refine and validate requirements, **use-case modeling** was employed to describe system interactions from the perspective of each user role. Preliminary **UI sketches and workflow storyboards** assisted in visualizing essential functions and receiving early stakeholder feedback. Additionally, reference to **existing school management processes** and sample academic data enabled the identification of practical constraints and operational rules.
- These combined techniques ensure that the resulting requirements are realistic, complete, and aligned with actual institutional needs, forming a reliable foundation for system design and development.

## 3.4 Functional Requirements

This section defines the **functional requirements** of the **School Management System (SMS)**. Functional requirements specify **what the system shall do** to support academic, administrative, and communication processes within a school environment.

Each requirement is stated in **clear, testable, and unambiguous terms** to ensure correct interpretation and verification. The requirements are **grouped by major system modules**, including Authentication, Student Management, Teacher Operations, Parent Interaction, Administration, and System Services.

### User Authentication and Authorization

#### *FR-1: User Login & Role-Based Access*

*Table 6 Defines the system requirements for secure user authentication and redirection to role-specific dashboards.*

| | |
|---|---|
| **Identifier** | FR-1 |
| **Title** | User Authentication & Role-Based Routing |
| **Requirement** | The system shall authenticate users using email/username and password, and route them to a role-specific dashboard based on their assigned role (Admin, Teacher, Student, Parent). |
| **Source** | Project Proposal, Stakeholder Interviews (Teachers, Admin Staff) |
| **Rationale** | Ensures secure access and tailored system functionality. |
| **Business Rule (if required)** | • Only verified users with an active status may log in.<br>• Each user account must be associated with exactly one role at a time. |

| | |
|---|---|
| | • Password must meet school-defined security standards (e.g., minimum length, complexity).<br>• Users with inactive or suspended accounts shall not be allowed to access the system. |
| **Dependencies** | None |
| **Priority** | High |

**Error Handling / Invalid Input Response:**

- If incorrect credentials are entered → system displays *"Invalid username or password"*.
- If account is inactive → system displays *"Your account is deactivated. Contact Admin."*
- If password does not meet requirements → reject and prompt valid format.
- System must block login after repeated failed attempts (security rule).

### *FR-2: User Management*

*Table 7 Specifies administrative capabilities for creating, updating, deactivating, and managing user accounts and roles.*

| | |
|---|---|
| **Identifier** | FR-2 |
| **Title** | User Management by Admin |
| **Requirement** | The Admin shall be able to add, update, deactivate, or delete user accounts and assign appropriate roles. |
| **Source** | Admin Requirements, Proposal |
| **Rationale** | Ensures proper system access control and allows accurate management of school personnel and students. |
| **Business Rule (if required)** | • Only Admin can create or remove accounts.<br>• Roles must be assigned at account creation. |
| **Dependencies** | FR-1 |
| **Priority** | High |

**Error Handling / Invalid Input Response:**

- If mandatory fields (name, role, email) are missing → show validation error.

- If Admin tries to delete a user with existing linked records → system shall prevent deletion and show error.
- If duplicate email/username exists → system shall reject the account creation.

## Attendance Management

### *FR-3: Mark Attendance*

*Table 8 Describes the functionality that allows teachers to record daily student attendance for assigned classes.*

| | |
|---|---|
| **Identifier** | FR-3 |
| **Title** | Mark Attendance |
| **Requirement** | Teachers shall be able to mark daily attendance for students in assigned classes using Present/Absent/Late options. |
| **Source** | Teacher Interviews, Proposal |
| **Rationale** | Attendance is critical for academic tracking and portfolio generation. |
| **Business Rule (if required)** | • Only assigned teachers can mark attendance for their classes.<br>• Attendance must be recorded once per day per class. |
| **Dependencies** | FR-1, FR-2 |
| **Priority** | High |

**Error Handling / Invalid Input Response:**

- If a teacher attempts to mark attendance for a class not assigned → system shows *"Unauthorized action."*
- If attendance already submitted → system shows *"Attendance already marked for today."*
- If internet/server interruption occurs → partial entries should be autosaved (if feasible).

## Marks and Student Performance

### *FR-4: Enter Marks*

*Table 9 Defines the requirements for teachers to enter and manage student marks for tests and examinations.*

| | |
|---|---|
| **Identifier** | FR-4 |
| **Title** | Enter Marks |
| **Requirement** | Teachers shall be able to enter marks for tests, and exams for each student in their subjects. |
| **Source** | Teacher Input, Proposal |
| **Rationale** | Marks entry is vital for grading, performance evaluation, and portfolio compilation. |
| **Business Rule (if required)** | • Marks must be numeric.<br>• Marks may not exceed defined maximum for each exam/test.<br>• Only subject-assigned teachers may enter marks. |
| **Dependencies** | FR-2, FR-1 |
| **Priority** | High |

**Error Handling / Invalid Input Response:**

- If marks exceed allowed range → system rejects input.
- If teacher attempts to enter marks for unassigned subjects → system shows *"Access denied."*
- If marks are missing for required fields → system prompts completion.

### *FR-5: Achievements Upload*

*Table 10 Specifies the process for uploading and managing student achievements and certificates as part of the digital portfolio.*

| | |
|---|---|
| **Identifier** | FR-5 |
| **Title** | Achievements Upload |

| | |
|---|---|
| **Requirement** | Teachers shall be able to upload student achievements, certificates, awards, and extracurricular activity records to be included automatically in the student portfolio. |
| **Source** | Project Proposal – Portfolio Generator |
| **Rationale** | Supports a complete digital portfolio by storing all student accomplishments in one centralized location. |
| **Business Rule (if required)** | • Only Teachers assigned to the student's class can upload achievements.<br>• Uploads must include title, date, category (Academic / Sports / Co-curricular / Certificates)<br>• Uploaded files must follow allowed size and type constraints.<br>• Students/Parents may view uploaded achievements but cannot modify them. |
| **Dependencies** | FR-2, FR-1 |
| **Priority** | High |

**Error Handling / Invalid Input Response:**

- If file size exceeds limit → system refuses upload and shows *"File too large."*
- If file type is invalid → system displays allowed formats.
- If teacher tries uploading for unassigned student → access denied.
- If upload fails due to server error → display *"Upload failed. Try again."* and log the error.

### *FR-6: Behavior remarks by teachers*

*Table 11 Describes how teachers can record and manage student behaviour remarks for academic and disciplinary tracking.*

| | |
|---|---|
| **Identifier** | FR-6 |
| **Title** | Behavior Remarks by Teachers |
| **Requirement** | Teachers shall be able to record positive or negative behaviour remarks for each student, which will become part of the student's behaviour history and portfolio. |
| **Source** | Stakeholder Inputs – Behaviour & Complaint Management |

| | |
|---|---|
| **Rationale** | Provides a structured and formal way to track student behaviour trends, disciplinary issues, and positive contributions. |
| **Business Rule (if required)** | • Only assigned teachers may add behaviour remarks for their students.<br>• Behaviour remarks automatically appear in the student's portfolio and may influence risk level.<br>• teachers can only edit their remarks. |
| **Dependencies** | FR-2, FR-1 |
| **Priority** | High |

**Error Handling / Invalid Input Response:**

- If mandatory fields (description/category) are missing → system shows validation error.
- If unauthorized user tries adding remarks → system denies action.
- If remark exceeds character limit → system prompts trimming.
- If behavior entry clashes with missing student record → system shows *"Student not found."*

### *FR-7: Student Risk Monitoring*

*Table 12 Defines automated analysis of attendance, marks, and behaviour to generate student risk indicators.*

| | |
|---|---|
| **Identifier** | FR-7 |
| **Title** | Student Risk Monitoring |
| **Requirement** | The system shall automatically analyze student attendance, marks, and behaviour records to generate risk indicators (Low Risk, Medium Risk, High Risk) |
| **Source** | Project Proposal – Risk Monitoring Objective |
| **Rationale** | Helps teachers and administration identify academically or behaviourally struggling students early and take corrective actions. |
| **Business Rule (if required)** | • Risk levels must be calculated based on predefined criteria (e.g., low attendance, poor marks, repeated complaints).<br>• Risk indicators must update automatically when new data (attendance/marks/complaints) is added. |

| | |
|---|---|
| | • Only Admins and Teachers may view full risk analysis; Parents see only summarized indicators (if enabled). |
| **Dependencies** | FR-3, FR-4 |
| **Priority** | High |

**Error Handling / Invalid Input Response:**

- If required data is missing → system shows *"Insufficient data to generate risk level."*
- If unauthorized user tries accessing the module → access denied.
- If risk calculation fails due to server error → log issue and display fallback status *"Risk unavailable"*.

## Student Portfolio Generation

### *FR-8: Portfolio Compilation*

*Table 13 Specifies the system's ability to compile attendance, marks, behaviour, and achievements into a unified student portfolio.*

| | |
|---|---|
| **Identifier** | FR-8 |
| **Title** | Portfolio Compilation |
| **Requirement** | The system shall compile student attendance, marks, behavior notes, and achievements into a structured digital portfolio. |
| **Source** | Proposal |
| **Rationale** | Provides a consolidated view of student performance. |
| **Business Rule (if required)** | • Marks must be numeric .<br>• Only relevant stakeholders can generate/view portfolios. |
| **Dependencies** | FR-3, FR-4 |
| **Priority** | Medium |

**Error Handling / Invalid Input Response:**

- If any required data is missing, system shows *"Portfolio incomplete—missing attendance/marks/etc."*
- If unauthorized user tries to access portfolio → deny access.

## Noticeboard & Notifications

### *FR-9: Post Notices*

*Table 14 Describes the functionality for authorized staff to publish notices to students and parents.*

| | |
|---|---|
| **Identifier** | FR-9 |
| **Title** | Post Notices |
| **Requirement** | Admin/Teachers shall be able to post school-wide or class-specific notices visible on user dashboards. |
| **Source** | Admin |
| **Rationale** | Improves communication across all stakeholders. |
| **Business Rule (if required)** | • Only authorized staff can create notices.<br>• Notice may be tagged as school-wide or class-specific. |
| **Dependencies** | FR-9 |
| **Priority** | High |

**Error Handling / Invalid Input Response:**

- If mandatory content is missing → system refuses publishing.
- If unauthorized user attempts posting → show *"Permission denied."*

### *FR-10: Notify Users*

*Table 15 Defines the notification mechanism for informing users about notices and important system updates.*

| | |
|---|---|
| **Identifier** | FR-10 |
| **Title** | Notify Users |
| **Requirement** | The system shall notify parents and students about new notices via dashboard alerts |
| **Source** | Parent Feedback |
| **Rationale** | Ensures timely communication of important announcements. |
| **Business Rule (if required)** | • Notifications shall follow user-specific communication settings. |

| | |
|---|---|
| | • Notifications must be generated instantly after posting a notice. |
| **Dependencies** | FR-6 |
| **Priority** | Medium |

**Error Handling / Invalid Input Response:**

- If notification delivery fails → system logs the failure for Admin review.
- If user disables notifications → system respects settings.

## Parent–Teacher Messaging

### *FR-11: Private Messaging*

*Table 16 Specifies secure communication requirements between parents and teachers within the system.*

| | |
|---|---|
| **Identifier** | FR-11 |
| **Title** | Private Messaging |
| **Requirement** | The system shall allow parents and teachers to exchange private messages through a secure messaging interface. |
| **Source** | Parent–Teacher Meeting Notes |
| **Rationale** | Improves communication and student monitoring. |
| **Business Rule (if required)** | • Messaging only allowed between assigned parent ↔ teacher pairs.<br>• Messages must be stored securely for audit. |
| **Dependencies** | FR-1, FR2 |
| **Priority** | High |

**Error Handling / Invalid Input Response:**

- If unauthorized user attempts to message → access denied.
- If message body is empty → system rejects sending.

## Complaint and Behavior Management

### *FR-12: Submit Complaint*

*Table 17 Defines the process through which students and parents can submit formal complaints.*

| | |
|---|---|
| **Identifier** | FR-12 |
| **Title** | Submit Complaint |
| **Requirement** | Students/Parents shall be able to submit complaints |
| **Source** | Student Surveys |
| **Rationale** | Provides a formal channel for academic or behavioral concerns. |
| **Business Rule (if required)** | • Complaints shall be categorized (e.g., academic, discipline). |
| **Dependencies** | FR-1 |
| **Priority** | Medium |

**Error Handling / Invalid Input Response:**

- If mandatory fields are missing → display error.
- If user is not authenticated → deny submission.

### *FR-13: Complaint Resolution Workflow*

*Table 18 Describes how teachers and administrators manage and resolve submitted complaints.*

| | |
|---|---|
| **Identifier** | FR-13 |
| **Title** | Complaint Resolution Workflow |
| **Requirement** | Teachers/Admin shall update complaint status (Pending, In Review, Resolved) |
| **Source** | Admin Decision |
| **Rationale** | Ensures transparent complaint handling. |
| **Business Rule (if required)** | • Only authorized staff may update status. |
| **Dependencies** | FR-9 |

| | |
|---|---|
| **Priority** | Medium |

**Error Handling / Invalid Input Response:**

- If unauthorized staff attempts update → system denies access.
- If complaint does not exist → show "Complaint not found."

## Fee Reminder

### *FR-14: Fee Status View*

*Table 19 Specifies the system requirements for displaying student fee status to authorized users.*

| | |
|---|---|
| **Identifier** | FR-14 |
| **Title** | Fee Status View |
| **Requirement** | The system shall display the fee status (Paid/Unpaid/Pending) for each student. |
| **Source** | Finance Department |
| **Rationale** | Increases transparency for parents and students. |
| **Business Rule (if required)** | • Only Admin can update fee records.<br>• Students/Parents can only view their own fee data. |
| **Dependencies** | FR-1 |
| **Priority** | Medium |

**Error Handling / Invalid Input Response:**

- If user tries to view another student's fee → access denied.
- If fee data is missing → show "Fee record not available."

## Audit Logs & System Monitoring

### *FR-15: Audit Logging*

*Table 20 Defines the system's logging requirements for tracking critical actions and ensuring accountability.*

| | |
|---|---|
| **Identifier** | FR-15 |
| **Title** | Audit Logging |
| **Requirement** | The system shall record logs for login activity, attendance marking, marks entry, complaint updates, and account changes. |
| **Source** | Admin Security Policy |
| **Rationale** | Supports accountability and system monitoring. |
| **Business Rule (if required)** | • Logs must be kept for at least one year.<br>• Logs shall be immutable. |
| **Dependencies** | All functional modules |
| **Priority** | High |

**Error Handling / Invalid Input Response:**

- If logging fails → system shows an Admin alert and stores error in backup log.
- Unauthorized access to logs → strictly denied.

# 3.5 Non-Functional Requirements

This section specifies the **nonfunctional requirements (NFRs)** of the **School Management System (SMS)**, which define **how the system should perform** rather than specific functionalities. These requirements address system **quality attributes**, **performance standards**, **security**, **usability**, **reliability**, **scalability**, and **maintainability**.

## 1. Reliability

**NFR-R1**: System Uptime

The system shall maintain at least 98% uptime during school operational hours (7 AM – 6 PM).

**NFR-R2**: Backup Frequency

The system shall automatically create an incremental database backup every 24 hours at 9 PM, ensuring data protection.

**NFR-R3**: Data Integrity

The system shall ensure 99% data integrity, meaning records for attendance, marks, notices, and communication must not be lost or corrupted during CRUD operations.

**NFR-R4**: Recovery Time Objective (RTO)

In case of system failure, the system shall recover within 10 minutes after restoration procedures begin.

**NFR-R5**: Fault Tolerance

If a module fails (e.g., noticeboard service), the rest of the system shall continue working without interruption.

### Usability

**NFR-U1**: Learning Curve

A new user (teacher/parent/student) shall be able to learn the system interface within 30 minutes without training.

**NFR-U2**: Task Completion

- A teacher shall be able to mark attendance in 3 clicks or fewer.
- A parent shall be able to check the student portfolio within 4 clicks.
- A student shall be able to submit a complaint within 3 clicks.

**NFR-U3**: Accessibility

The system shall be fully accessible on mobile, tablet, and desktop with responsive design (React + Tailwind/Bootstrap).

**NFR-U4**: Error Prevention

User input errors (e.g., missing required fields) shall be detected and user-friendly error messages displayed within 1 second.

**NFR-U5**: Language & Clarity

The interface shall maintain consistent terminology and iconography for all dashboards (Students, Teachers, Parents, Admin).

## 2. Performance

**NFR-P1**: Response Time

For all CRUD operations (attendance entry, portfolio view, noticeboard load, messaging), the system shall respond within:

- 2 seconds for simple operations
- 5 seconds for complex queries (e.g., portfolio generation)

**NFR-P2**: Concurrent Users

The system shall support 100 concurrent users without significant degradation in performance (since it is designed for a single school environment).

**NFR-P3**: Data Processing

The system shall process a full class attendance record (40–60 students) within 3 seconds.

**NFR-P4**: Page Rendering

95% of all pages shall load in under 3 seconds on a standard 4G connection.

## 3. Security

**NFR-S1**: Role-Based Access Control

The system shall enforce strict role-based access to protect sensitive data for Students, Teachers, Parents, and Admins.

**NFR-S2**: Unauthorized Access Prevention

Any unauthorized attempt to access another user's data shall be blocked and logged within 1 second.

**NFR-S3**: Data Protection

All sensitive data (student profiles, marks, attendance, complaints) shall meet basic confidentiality and integrity standards (no plaintext exposure).

**NFR-S4**: Session Management

User sessions shall automatically expire after 20 minutes of inactivity and require reauthentication.

**NFR-S5**: Audit Trail

The system shall maintain audit logs for:

- Attendance updates
- Marks entry
- Portfolio updates

- Complaint status changes
- Notice creation
- Each log shall include timestamp, user role, and action details.

**NFR-S6**: Communication Privacy

The Parent–Teacher messaging module shall ensure:

- Only sender and receiver can view messages
- No public or group message visibility.

# 3.6 External Interface Requirements

This section defines all interfaces through which the **School Management System (SMS)** interacts with its **users, external hardware, external software components, communication mechanisms, and supporting platforms.** These requirements ensure smooth communication, consistent user experience, and correct integration with external technologies.

## 1. User Interfaces Requirements

The School Management System is a web-based application used by four primary user roles: **Admin, Teacher, Student, and Parent**. The following logical user interface requirements apply to all modules.

### GUI Design Standards

The system shall follow consistent UI/UX standards across all dashboards, including:

- Alignment with **Google Material Design** and **React component styling conventions**.
- Use of **Bootstrap or Tailwind CSS** for responsive and mobile-friendly layouts.
- Consistent use of fonts, icons, layouts, and interactive components across all pages.
- Standard button labels such as **Save, Update, Submit, Cancel, Back, Upload**, and **Delete**.
- A uniform **color scheme** representing institutional or educational themes (blue, green, white).
- A minimum screen resolution compatibility of **1366 × 768**, with support for responsive adaptation to mobile devices.

### Navigation & Controls

- A left-side or top navigation bar shall be present on every dashboard with clearly labeled links.
- Common navigation items include: **Home, Notices, Messages, Attendance, Portfolio, and Logout**.
- A **Help** or **Support** button will be accessible from every screen.

- Consistent placement of **Back**, **Home**, and **Dashboard** icons.
- Tab/field navigation shall follow left-to-right logical movement for form-based inputs.
- Forms shall support error highlighting, placeholder text, and tooltip hints where necessary.

### Display Conventions

- Success messages shall appear in green; error or warning messages in red.
- System messages shall appear in toast alerts or modal dialogs.
- Required fields shall be marked with a red asterisk (*).
- Data tables shall support pagination, filtering, and search.
- Icons shall be used for common actions (edit pencil, delete bin, upload cloud, etc.).

### Accessibility Requirements

- The interface shall comply with **WCAG 2.0 Level AA** accessibility guidelines.
- High-contrast mode shall be supported for users with visual impairments.
- Provide text alternatives for icons and images using **alt-text** attributes.
- Font size shall be adjustable using browser zoom (up to 200%).
- All buttons and links shall be keyboard-accessible (TAB, ENTER, SHIFT+TAB).

### Localization & Language Support

- English shall be the primary interface language.
- The system shall allow easy integration of additional languages (e.g., Urdu) by using:
    1) Unicode-supported fonts
    2) Separated label constants
    3) Right-to-left layout support (if Urdu is added)

### Mockups & UI Documentation

- Screen mockups and wireframes shall be provided in a **separate User Interface Design Document**.
- Mockups included in the SRS serve only as illustrative examples and not final design.

## 2. Software interfaces

This section describes all interactions between the SMS and external software systems, libraries, frameworks, and databases.

### Backend Framework

- The system shall use **ASP.NET Core Web API v7.0+** for server-side logic.
- API endpoints shall follow RESTful conventions (GET, POST, PUT, DELETE).
- Authentication shall be implemented using **JWT (JSON Web Tokens)**.

### Frontend Framework

- The frontend shall be built using **React.js v18+**.
- Axios or Fetch API shall be used for server communication.

### Database Interface

- The system shall use **Microsoft SQL Server 2019+** as the primary database.
- Database interactions shall occur through:
    1) **Entity Framework Core v7.0+** (ORM)
    2) Code-first migrations for schema generation
- SQL Server must support:
    1) User tables
    2) Attendance records
    3) Marks records
    4) Complaints and behaviour logs
    5) Portfolio data
    6) Audit logs

### External Libraries

The system shall integrate the following libraries:

- **JWT Authentication Library** for secure login
- **React Router** for navigation
- **React Icons / Lucide Icons** for visual consistency
- **Tailwind / Bootstrap** for UI layouts
- **Chart.js or Recharts** for analytics dashboards

### External Systems (Optional Integrations)

If enabled:

- **Email service provider (SMTP)** for sending alerts
- **SMS Gateway API** for fee reminders and notices
  These interfaces will require:
  1) Valid API key
  2) Network availability
  3) Compliance with security policies

## 3. Hardware interfaces

The School Management System does not require specialized hardware. It must operate on standard computing devices meeting the following requirements:

### Supported Devices

- Desktop computers
- Laptops
- Tablets
- Mobile phones

All devices must support:

- A modern web browser
- Minimum 2 GB RAM (recommended 4 GB for teachers/admin due to dashboards)

### Server Hardware

If self-hosted, the backend server shall have:

- Minimum **8 GB RAM**
- **100 GB storage**
- **Quad-core processor**
- Windows Server or Linux environment

### Hardware Integration

No direct hardware integrations (barcode scanners, biometric devices, etc.) are included *in current scope.*

System architecture allows future extensions.

## 4. Communications interfaces

This section describes all communication protocols and network interactions of the SMS.

### Network Protocols

The system shall communicate over:

- **HTTPS** (mandatory)
- **TLS 1.2+ encryption**
- **RESTful API calls** over port 443

### Web Browser Compatibility

The system must support:

- Google Chrome
- Microsoft Edge
- Firefox

- Safari (for iOS)

### Messaging Communication

- Parent–Teacher messaging shall use encrypted API calls.
- All messages shall be stored securely in the database.

### API Communication

- All client-server communication shall follow JSON formatting.
- API responses must include:
  1) Status code
  2) Success/Failure message
  3) Error details where applicable

### System Availability Requirements

- The system shall be operable with internet speeds as low as **2 Mbps**.
- Requests should time out only after 30 seconds of inactivity.

## 3.7 Use case Analysis

### 1. UC1 – User Login & Role-Based Access

*Table 21 UC1 – User Login & Role-Based Access – Authenticate users and route to role-specific dashboards*

| | |
|---|---|
| ***UC Identifier*** | UC1 |
| ***Requirements Traceability*** | FR-1 |
| ***Purpose*** | Authenticate users and route them to role-specific dashboards |
| ***Priority*** | High |
| ***Preconditions*** | User account exists and is active |
| ***Postconditions*** | User logged in and redirected to dashboard |
| ***Actors*** | Student, Parent, Teacher, Admin |
| ***Extends*** | None |
| ***Main Success Scenario*** | 1. User enters credentials<br>2. System verifies credentials<br>3. Checks user role<br>4. Redirects to role-specific dashboard<br>5. Dashboard displays widgets |

| | |
|---|---|
| ***Alternate Flows*** | Password expired → prompt resetInactive user → show deactivated |
| ***Exceptions*** | Invalid credentials → errorAccount temporarily locked |
| ***Includes*** | UC24 |

## 2. UC2 – Account Management (Admin)

*Table 22 UC2 – Account Management (Admin) – Enable administrators to create, update, deactivate, or delete user accounts and assign system roles.*

| | |
|---|---|
| ***UC Identifier*** | UC2 |
| ***Requirements Traceability*** | FR-2 |
| ***Purpose*** | Admin creates, updates, deactivates, or deletes user accounts and assigns roles |
| ***Priority*** | High |
| ***Preconditions*** | Admin logged in (UC1) |
| ***Postconditions*** | Accounts created/updated/deactivated successfully |
| ***Actors*** | Admin |
| ***Extends*** | None |
| ***Main Success Scenario*** | 1. Admin navigates to User Management<br>2. Selects “Add User”<br>3. Enters details and assigns role<br>4. System validates and creates account5. Action logged |
| ***Alternate Flows*** | Admin updates existing user / deactivates |
| ***Exceptions*** | Duplicate email/username → errorMandatory fields missing |
| ***Includes*** | UC3, UC24 |

## 3. UC3 – Parent Account Verification

*Table 23 UC3 – Parent Account Verification – Verify parent accounts and activate secure login credentials.*

| | |
|---|---|
| ***UC Identifier*** | UC3 |
| ***Requirements Traceability*** | FR-1, FR-2 |
| ***Purpose*** | Verify parent accounts and activate login credentials |

| | |
|---|---|
| ***Priority*** | High |
| ***Preconditions*** | Parent account exists (created via UC2) |
| ***Postconditions*** | Parent account activated and credentials verified |
| ***Actors*** | Admin, Parent |
| ***Extends*** | None |
| ***Main Success Scenario*** | 1. Admin sends activation link2. Parent sets password3. System validates4. Account activated |
| ***Alternate Flows*** | Parent requests resend / manual reset |
| ***Exceptions*** | Expired activation link → prompt new linkIncorrect password setup |
| ***Includes*** | UC1, UC24 |

## 4. UC4 – Dashboard View

*Table 24 UC4 – Dashboard View – Display a role-based dashboard with relevant widgets, shortcuts, and system information.*

| | |
|---|---|
| ***UC Identifier*** | UC4 |
| ***Requirements Traceability*** | FR-4, FR-5, FR-9, FR-10, FR-14 |
| ***Purpose*** | Provide role-based dashboard with relevant widgets and shortcuts |
| ***Priority*** | High |
| ***Preconditions*** | User logged in (UC1) |
| ***Postconditions*** | Dashboard displays correct widgets and links based on role |
| ***Actors*** | Student, Parent, Teacher, Admin |
| ***Extends*** | None |
| ***Main Success Scenario*** | 1. System loads dashboard<br>2. Displays widgets (attendance, marks, portfolio, fee status, notifications)<br>3. Navigation to modules enabled |
| ***Alternate Flows*** | Widgets may hide sections if no data |
| ***Exceptions*** | System fails to load → retry option |
| ***Includes*** | UC5, UC6, UC7, UC10, UC14, UC16 |

## 5. UC5 – View Attendance (Student)

*Table 25 UC5 – View Attendance (Student) – Allow students to view subject-wise attendance records and trends.*

| | |
|---|---|
| ***UC Identifier*** | UC5 |
| ***Requirements Traceability*** | FR-3 |
| ***Purpose*** | Allow students to view attendance records and trends |
| ***Priority*** | High |
| ***Preconditions*** | Student logged in (UC1) and enrolled in classes |
| ***Postconditions*** | Attendance data displayed with calendar and trend graphs |
| ***Actors*** | Student |
| ***Extends*** | UC4 |
| ***Main Success Scenario*** | 1. Student selects "Attendance"<br>2. System shows calendar with status<br>3. Bar graph trends<br>4. Option to download report |
| ***Alternate Flows*** | View previous months' attendance |
| ***Exceptions*** | Data unavailable → show message |
| ***Includes*** | UC14 |

## 6. UC6 – Mark Attendance (Teacher)

*Table 26 UC6 – Mark Attendance (Teacher) – Allow teachers to record daily attendance for assigned classes.*

| | |
|---|---|
| ***UC Identifier*** | UC6 |
| ***Requirements Traceability*** | FR-3 |
| ***Purpose*** | Teachers record daily attendance for assigned classes |
| ***Priority*** | High |
| ***Preconditions*** | Teacher logged in (UC1) and assigned classes |
| ***Postconditions*** | Attendance recorded; notifications sent to students/parents |
| ***Actors*** | Teacher |

| | |
|---|---|
| ***Extends*** | UC4 |
| ***Main Success Scenario*** | 1. Teacher selects class/section<br>2. Marks Present/Absent/Late<br>3. System validates entries<br>4. Records saved; notifications sent |
| ***Alternate Flows*** | Update attendance before submission deadline |
| ***Exceptions*** | Attempt to mark unassigned class → access deniedInternet interruption → autosave partial entries |
| ***Includes*** | UC14 |

## 7. UC7 – Enter Marks (Teacher)

*Table 27 UC7 – Enter Marks (Teacher) – Enable teachers to enter and manage student marks for exams and assessments.*

| | |
|---|---|
| ***UC Identifier*** | UC7 |
| ***Requirements Traceability*** | FR-4 |
| ***Purpose*** | Teachers enter marks for exams/assessments |
| ***Priority*** | High |
| ***Preconditions*** | Teacher logged in (UC1) and assigned subjects |
| ***Postconditions*** | Marks recorded; grades and remarks stored |
| ***Actors*** | Teacher |
| ***Extends*** | UC4 |
| ***Main Success Scenario*** | 1. Teacher selects exam and subject<br>2. Inputs marks per student<br>3. System validates and saves<br>4. Students/parents can view |
| ***Alternate Flows*** | Edit previously entered marks before final submission |
| ***Exceptions*** | Marks exceed max → rejectedUnauthorized class → denied |
| ***Includes*** | UC14 |

# 8. UC8 – Upload Achievements (Teacher)

*Table 28 UC8 – Upload Achievements (Teacher) – Allow teachers to upload student certificates and extracurricular achievements.*

| | |
|---|---|
| ***UC Identifier*** | UC8 |
| ***Requirements Traceability*** | FR-5 |
| ***Purpose*** | Teachers upload certificates and extracurricular records |
| ***Priority*** | High |
| ***Preconditions*** | Teacher logged in (UC1) and assigned students |
| ***Postconditions*** | Achievements stored in student portfolio; visible to student/parent |
| ***Actors*** | Teacher |
| ***Extends*** | UC4 |
| ***Main Success Scenario*** | 1. Teacher selects "Upload Achievements"<br>2. Uploads file with title, date, category<br>3. System validates and saves<br>4. Visible in portfolio |
| ***Alternate Flows*** | Edit title/remarks after upload; batch uploads |
| ***Exceptions*** | File type/size invalid → errorUnauthorized student → denied |
| ***Includes*** | UC14 |

# 9. UC9 – Record Behavior Remarks (Teacher)

*Table 29 UC9 – Record Behavior Remarks (Teacher) – Record positive or negative behavior remarks for students.*

| | |
|---|---|
| ***UC Identifier*** | UC9 |
| ***Requirements Traceability*** | FR-6 |
| ***Purpose*** | Record positive or negative student behavior remarks |
| ***Priority*** | High |
| ***Preconditions*** | Teacher logged in and assigned students |
| ***Postconditions*** | Behavior remarks saved; portfolio updated; risk monitoring triggered |
| ***Actors*** | Teacher |
| ***Extends*** | UC4 |

| | |
|---|---|
| ***Main Success Scenario*** | 1. Teacher selects student<br>2. Adds remarks (positive/negative)<br>3. System saves and updates portfolio/risk |
| ***Alternate Flows*** | Edit remarks within allowed period |
| ***Exceptions*** | Mandatory fields missing → validation errorUnauthorized teacher → denied |
| ***Includes*** | UC11, UC14 |

## 10. UC10 – View Student Portfolio (Student/Parent)

*Table 30 UC10 – View Student Portfolio (Student/Parent) – Provide students and parents access to a consolidated student portfolio.*

| | |
|---|---|
| ***UC Identifier*** | UC10 |
| ***Requirements Traceability*** | FR-8 |
| ***Purpose*** | Students/parents view consolidated student portfolio (attendance, marks, behavior, achievements) |
| ***Priority*** | Medium |
| ***Preconditions*** | Student/parent logged in (UC1); student data exists |
| ***Postconditions*** | Portfolio displays attendance, marks, behavior, achievements; downloadable/exportable |
| ***Actors*** | Student, Parent |
| ***Extends*** | UC4 |
| ***Main Success Scenario*** | 1. Student/parent selects “View Portfolio”<br>2. System compiles attendance, marks, behavior, achievements<br>3. Displays portfolio with PDF export option |
| ***Alternate Flows*** | Filter portfolio by semester/year |
| ***Exceptions*** | Missing data → “Portfolio incomplete” message |
| ***Includes*** | UC5, UC7, UC8, UC9 |

## 11. UC11 – Student Risk Monitoring (System)

*Table 31 UC11– Student Risk Monitoring (System) – Automatically analyze attendance, marks, and behavior to determine student risk levels.*

| | |
|---|---|
| ***UC Identifier*** | UC11 |
| ***Requirements Traceability*** | FR-7 |

| | |
|---|---|
| ***Purpose*** | System calculates risk level based on attendance, marks, behavior |
| ***Priority*** | High |
| ***Preconditions*** | Teacher/Admin logged in; student data exists |
| ***Postconditions*** | Risk indicators updated; viewable by stakeholders |
| ***Actors*** | Admin, Teacher, System |
| ***Extends*** | UC4 |
| ***Main Success Scenario*** | 1. System gathers attendance, marks, behavior<br>2. Computes Low/Medium/High risk<br>3. Updates dashboard/portfolio |
| ***Alternate Flows*** | Teacher sets manual overrides or notes |
| ***Exceptions*** | Insufficient data → "Risk unavailable"Unauthorized access → denied |
| ***Includes*** | UC5, UC7, UC9 |

## 12. UC12 – Submit Complaint (Student/Parent)

*Table 32 UC12 – Submit Complaint (Student/Parent) – Allow students and parents to submit formal complaints within the system.*

| | |
|---|---|
| ***UC Identifier*** | UC12 |
| ***Requirements Traceability*** | FR-12 |
| ***Purpose*** | Students/parents submit formal complaints |
| ***Priority*** | Medium |
| ***Preconditions*** | Student/parent logged in (UC1) |
| ***Postconditions*** | Complaint stored with status "Pending" |
| ***Actors*** | Student, Parent |
| ***Extends*** | UC4 |
| ***Main Success Scenario*** | 1. User selects "Submit Complaint"<br>2. Enters complaint details and category<br>3. Chooses relevant staff<br>4. System saves complaint |
| ***Alternate Flows*** | User edits complaint before assignment |
| ***Exceptions*** | Mandatory fields missing → validation errorUnauthorized submission → denied |
| ***Includes*** | UC13, UC14 |

## 13. UC13 – Resolve Complaint (Teacher/Admin)

*Table 33 UC13 – Resolve Complaint (Teacher/Admin) – Enable staff to review, manage, and resolve submitted complaints.*

| | |
|---|---|
| ***UC Identifier*** | UC13 |
| ***Requirements Traceability*** | FR-13 |
| ***Purpose*** | Staff review and resolve complaints |
| ***Priority*** | Medium |
| ***Preconditions*** | Complaint exists; teacher/admin logged in (UC1) |
| ***Postconditions*** | Complaint status updated; notifications sent |
| ***Actors*** | Teacher, Admin |
| ***Extends*** | UC12 |
| ***Main Success Scenario*** | 1. Staff selects complaint<br>2. Updates status (In Review/Resolved)<br>3. Adds remarks<br>4. System notifies student/parent |
| ***Alternate Flows*** | Admin reassigns complaint before resolution |
| ***Exceptions*** | Unauthorized access → denied Complaint not found |
| ***Includes*** | UC14 |

## 14. UC14 – Notify Users (System)

*Table 34 UC14 – Notify Users (System) – Send automated notifications for system events and updates.*

| | |
|---|---|
| ***UC Identifier*** | UC14 |
| ***Requirements Traceability*** | FR-10, FR-14 |
| ***Purpose*** | Send notifications for updates/events (marks, attendance, complaints, fee reminders) |
| ***Priority*** | High |
| ***Preconditions*** | Triggered by relevant events |
| ***Postconditions*** | Notifications delivered to intended recipients |
| ***Actors*** | System |
| ***Extends*** | None |

| | |
|---|---|
| ***Main Success Scenario*** | 1. Event occurs<br>2. System generates notification<br>3. Notification appears on dashboard/email/SMS |
| ***Alternate Flows*** | User disables notifications → respected |
| ***Exceptions*** | Delivery fails → logged for review |
| ***Includes*** | None |

## 15. UC15 – Post Notices (Teacher/Admin)

*Table 35 UC15 – Post Notices (Teacher/Admin) – Allow authorized staff to publish notices on the digital noticeboard.*

| | |
|---|---|
| ***UC Identifier*** | UC15 |
| ***Requirements Traceability*** | FR-9 |
| ***Purpose*** | Post notices to digital noticeboard |
| ***Priority*** | High |
| ***Preconditions*** | Teacher/Admin logged in (UC1) and authorized |
| ***Postconditions*** | Notice visible to users; notifications sent |
| ***Actors*** | Teacher, Admin |
| ***Extends*** | UC4 |
| ***Main Success Scenario*** | 1. User selects “Post Notice”<br>2. Enters title/content/type<br>3. Submits notice<br>4. System saves and triggers notifications |
| ***Alternate Flows*** | Schedule notice for future date |
| ***Exceptions*** | Mandatory fields missing → rejectedUnauthorized posting → denied |
| ***Includes*** | UC14 |

## 16. UC16 – Parent–Teacher Messaging

*Table 36 UC16 – Parent–Teacher Messaging – Enable secure private communication between parents and teachers.*

| | |
|---|---|
| ***UC Identifier*** | UC16 |
| ***Requirements Traceability*** | FR-11 |

| | |
|---|---|
| ***Purpose*** | Secure private messaging between parents and teachers |
| ***Priority*** | High |
| ***Preconditions*** | Parent/Teacher logged in (UC1); assigned pair exists |
| ***Postconditions*** | Message sent, received, and stored securely |
| ***Actors*** | Parent, Teacher |
| ***Extends*** | UC4 |
| ***Main Success Scenario*** | 1. User opens chat with assigned counterpart<br>2. Types and sends message<br>3. System saves and delivers message |
| ***Alternate Flows*** | Messages can include attachments; retry if delivery fails |
| ***Exceptions*** | Unauthorized messaging → deniedEmpty message → rejected |
| ***Includes*** | UC14 |

## 17. UC17 – View Fee Status (Student/Parent)

*Table 37 UC17 – View Fee Status (Student/Parent) – Allow students and parents to view fee status and download challans.*

| | |
|---|---|
| ***UC Identifier*** | UC17 |
| ***Requirements Traceability*** | FR-14 |
| ***Purpose*** | View fee payment status and download challans |
| ***Priority*** | Medium |
| ***Preconditions*** | Student/Parent logged in (UC1) |
| ***Postconditions*** | Fee information displayed; downloadable challans |
| ***Actors*** | Student, Parent |
| ***Extends*** | UC4 |
| ***Main Success Scenario*** | 1. User selects “Fee Status”<br>2. System retrieves student fee records<br>3. Displays summary<br>4. Option to download challan |
| ***Alternate Flows*** | User filters by semester or year |
| ***Exceptions*** | Attempt to view another student’s fees → deniedData missing → “Fee record not available” |
| ***Includes*** | UC14 |

# 18. UC18 – Manage Fees (Admin)

*Table 38 UC18 – Manage Fees (Admin) – Enable administrators to manage and update student fee records*

| | |
|---|---|
| ***UC Identifier*** | UC18 |
| ***Requirements Traceability*** | FR-14 |
| ***Purpose*** | Admin updates/adds/corrects student fee records |
| ***Priority*** | High |
| ***Preconditions*** | Admin logged in (UC1) |
| ***Postconditions*** | Fee records updated; notifications triggered |
| ***Actors*** | Admin |
| ***Extends*** | UC4 |
| ***Main Success Scenario*** | 1. Admin selects “Manage Fees”<br>2. Updates student fee status<br>3. System saves changes and triggers notification |
| ***Alternate Flows*** | Bulk fee updates via CSV import |
| ***Exceptions*** | Invalid fee amount → validation errorUnauthorized access → denied |
| ***Includes*** | UC14 |

# 19. UC19 – Manage Students (Admin)

*Table 39 UC19 – Manage Students (Admin) – Allow administrators to add, update, and manage student profiles.*

| | |
|---|---|
| ***UC Identifier*** | UC19 |
| ***Requirements Traceability*** | FR-3 |
| ***Purpose*** | Admin adds/updates/deactivates student profiles |
| ***Priority*** | High |
| ***Preconditions*** | Admin logged in (UC1) |
| ***Postconditions*** | Student profiles updated; records maintained |
| ***Actors*** | Admin |
| ***Extends*** | UC4 |

| | |
|---|---|
| ***Main Success Scenario*** | 1. Admin selects “Manage Students”<br>2. Adds/updates/deactivates record<br>3. Assigns class, section, roll number<br>4. System validates and saves |
| ***Alternate Flows*** | Bulk import via CSV |
| ***Exceptions*** | Duplicate roll number/email → rejectedMandatory fields missing → validation error |
| ***Includes*** | UC14 |

## 20. UC20 – Manage Teachers & Subjects (Admin)

*Table 40 UC20 – Manage Teachers & Subjects (Admin) – Enable administrators to manage teacher profiles and subject/class assignments.*

| | |
|---|---|
| ***UC Identifier*** | UC20 |
| ***Requirements Traceability*** | FR-4 |
| ***Purpose*** | Admin manages teacher profiles, assigns subjects, maps classes |
| ***Priority*** | High |
| ***Preconditions*** | Admin logged in (UC1) |
| ***Postconditions*** | Teacher profiles updated; assigned subjects/classes recorded |
| ***Actors*** | Admin |
| ***Extends*** | UC4 |
| ***Main Success Scenario*** | 1. Admin selects “Manage Teachers”<br>2. Adds/updates/deactivates profile<br>3. Assigns subjects/classes<br>4. System saves changes |
| ***Alternate Flows*** | Bulk upload of teachers |
| ***Exceptions*** | Duplicate email → rejectedUnauthorized access → denied |
| ***Includes*** | UC14 |

## 21. UC21 – Review Attendance & Marks (Admin)

*Table 41 UC21 – Review Attendance & Marks (Admin) – Admin reviews and corrects student attendance and marks.*

| | |
|---|---|
| ***UC Identifier*** | UC21 |
| ***Requirements Traceability*** | FR-5 |

| | |
|---|---|
| ***Purpose*** | Admin reviews and corrects attendance and marks |
| ***Priority*** | High |
| ***Preconditions*** | Admin logged in (UC1); data exists |
| ***Postconditions*** | Attendance/marks corrected; notifications triggered |
| ***Actors*** | Admin |
| ***Extends*** | UC4 |
| ***Main Success Scenario*** | 1. Admin selects "Review Attendance & Marks"<br>2. Applies corrections<br>3. System validates and saves<br>4. Notifications sent (UC14) |
| ***Alternate Flows*** | Partial data → system flags missing info |
| ***Exceptions*** | Unauthorized access → deniedData validation fails |
| ***Includes*** | UC14 |

## 22. UC22 – Compile Student Portfolio (System)

*Table 42 UC22 – Compile Student Portfolio (System) – System compiles attendance, marks, behavior, and achievements into a portfolio.*

| | |
|---|---|
| ***UC Identifier*** | UC22 |
| ***Requirements Traceability*** | FR-8 |
| ***Purpose*** | Compile attendance, marks, behavior, achievements into portfolio |
| ***Priority*** | Medium |
| ***Preconditions*** | Relevant data exists |
| ***Postconditions*** | Portfolio generated; available for viewing/export |
| ***Actors*** | System |
| ***Extends*** | UC10 |
| ***Main Success Scenario*** | 1. System gathers attendance, marks, behavior, achievements<br>2. Compiles portfolio<br>3. Option to export PDF provided |
| ***Alternate Flows*** | Partial data → system flags missing info |
| ***Exceptions*** | Missing data → "Portfolio incomplete" |
| ***Includes*** | UC14 |

## 23. UC23 – Password Reset / Change

*Table 43 UC23 –Password Reset / Change – Users can securely reset or change their password.*

| | |
|---|---|
| ***UC Identifier*** | UC23 |
| ***Requirements Traceability*** | FR-1 |
| ***Purpose*** | Reset or change password securely |
| ***Priority*** | High |
| ***Preconditions*** | User logged in |
| ***Postconditions*** | Password updated securely |
| ***Actors*** | Student, Parent, Teacher, Admin |
| ***Extends*** | None |
| ***Main Success Scenario*** | 1. User selects "Reset/Change Password"<br>2. System verifies identity<br>3. Password updated securely |
| ***Alternate Flows*** | Forgotten password → email verification link |
| ***Exceptions*** | Verification fails → cannot reset |
| ***Includes*** | Standalone; included by UC1, UC2, UC3 |

## 24. UC24 – Audit Logging (System)

*Table 44 UC24 –Audit Logging (System) – System maintains immutable logs of all critical actions.*

| | |
|---|---|
| ***UC Identifier*** | UC24 |
| ***Requirements Traceability*** | FR-15 |
| ***Purpose*** | Maintain audit logs for all critical system actions to ensure accountability and traceability. |
| ***Priority*** | High |
| ***Preconditions*** | User performs any tracked action (login, attendance, marks, portfolio updates, complaints, notices). |
| ***Postconditions*** | Logs created with timestamp, user role, and action details. |
| ***Actors*** | System |
| ***Extends*** | None |

| | |
|---|---|
| ***Main Success Scenario*** | 1. User performs an action.<br>2. System records the event in audit logs.<br>3. Logs are stored securely and made immutable. |
| ***Alternate Flows*** | Logging service unavailable → queue logs and save once restored. |
| ***Exceptions*** | Unauthorized access attempt → denied and logged. |
| ***Includes*** | None |

## 25. UC25 – Generate Reports (Admin)

*Table 45 UC25 –Generate Reports (Admin) – Admin generates academic and administrative reports in PDF/CSV format.*

| | |
|---|---|
| ***UC Identifier*** | UC25 |
| ***Requirements Traceability*** | FR-23, FR-24, FR-7, FR-22 |
| ***Purpose*** | Enable admin to generate attendance, marks, portfolio, and student risk reports in PDF/CSV format. |
| ***Priority*** | High |
| ***Preconditions*** | Admin logged in (UC1); relevant data exists. |
| ***Postconditions*** | Reports generated and downloadable/printable. |
| ***Actors*** | Admin |
| ***Extends*** | UC4 (Dashboard View) |
| ***Main Success Scenario*** | 1. Admin selects report type.<br>2. Chooses filters (class, term, student).<br>3. System generates report.<br>4. Admin downloads or prints report. |
| ***Alternate Flows*** | Partial data → report flags missing information. |
| ***Exceptions*** | Report generation fails → error displayed; log updated. |
| ***Includes*** | UC24 (Audit Logging) |

## 26. UC26 – Logout (User)

*Table 46 UC26 – Logout (User) – Users can securely log out of the system.*

| | |
|---|---|
| ***UC Identifier*** | UC26 |
| ***Requirements Traceability*** | FR-1 (Security) |
| ***Purpose*** | End user session securely and redirect to login page. |

| | |
|---|---|
| ***Priority*** | High |
| ***Preconditions*** | User is logged in. |
| ***Postconditions*** | Session terminated; action logged. |
| ***Actors*** | Student, Parent, Teacher, Admin |
| ***Extends*** | None |
| ***Main Success Scenario*** | 1. User clicks “Logout.”<br>2. System invalidates session.<br>3. Redirects to login page. |
| ***Alternate Flows*** | None |
| ***Exceptions*** | Session termination fails → system alerts user and retries |
| ***Includes*** | UC24 (Audit Logging) |

# 27. Use Case Diagram

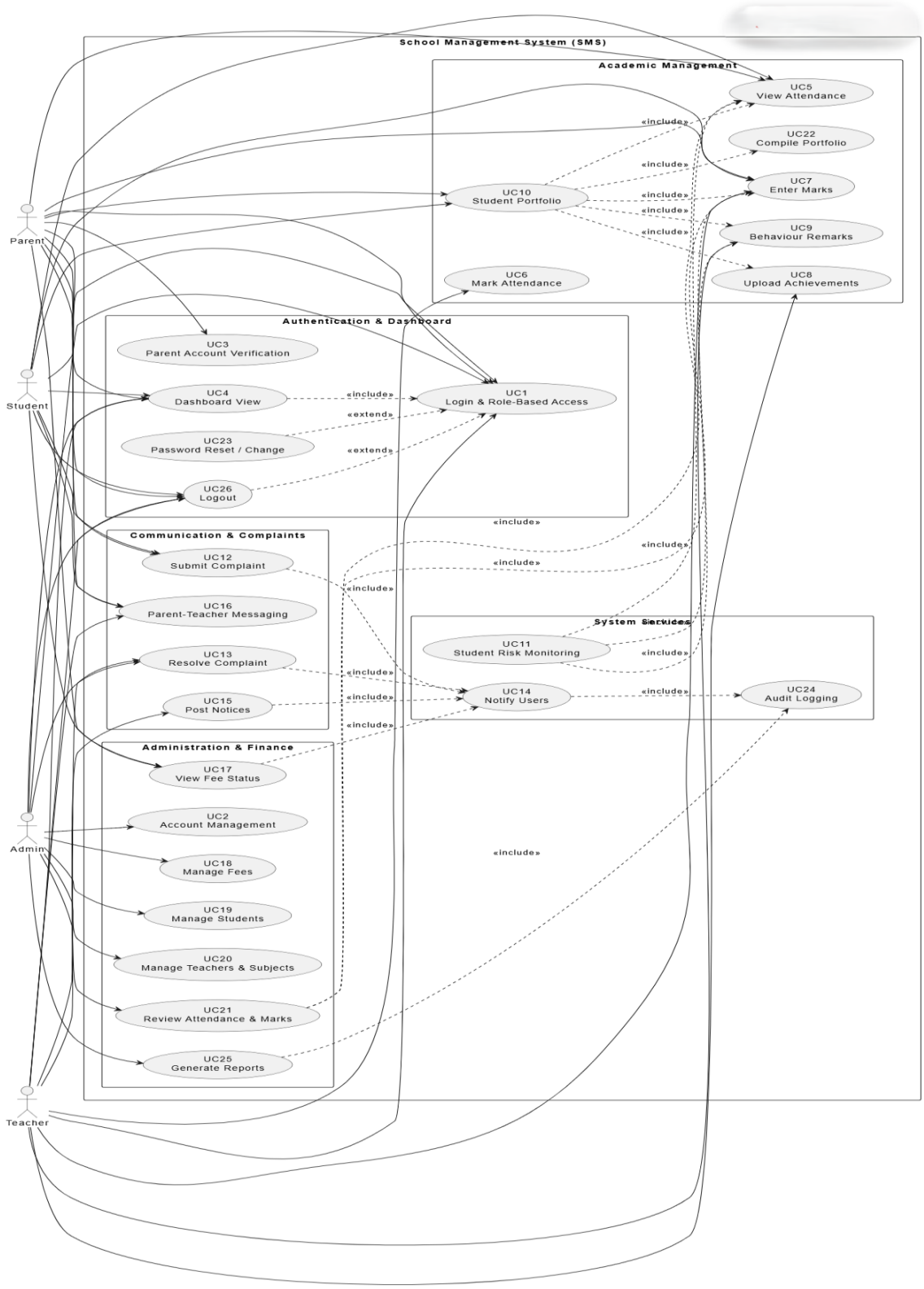
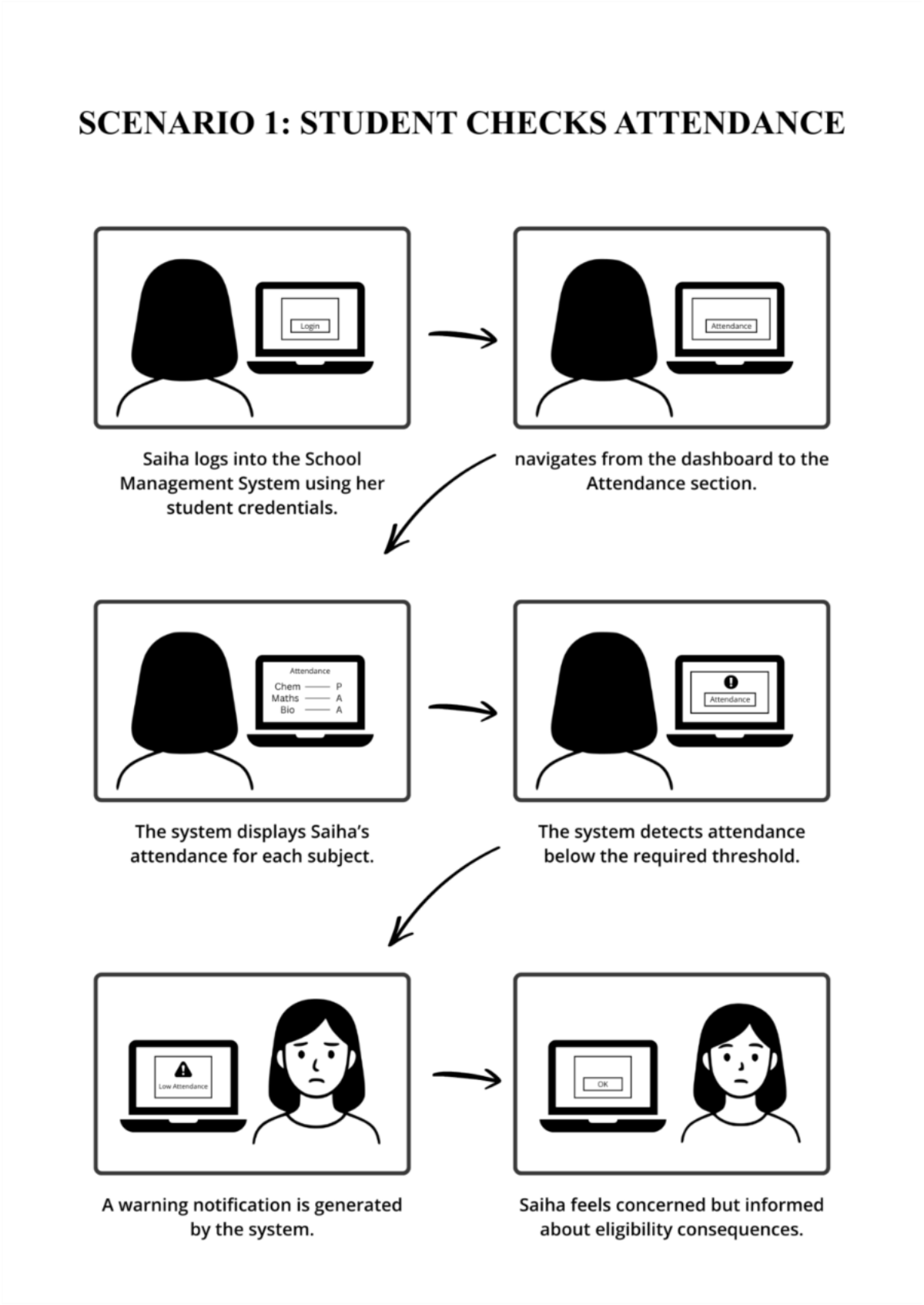
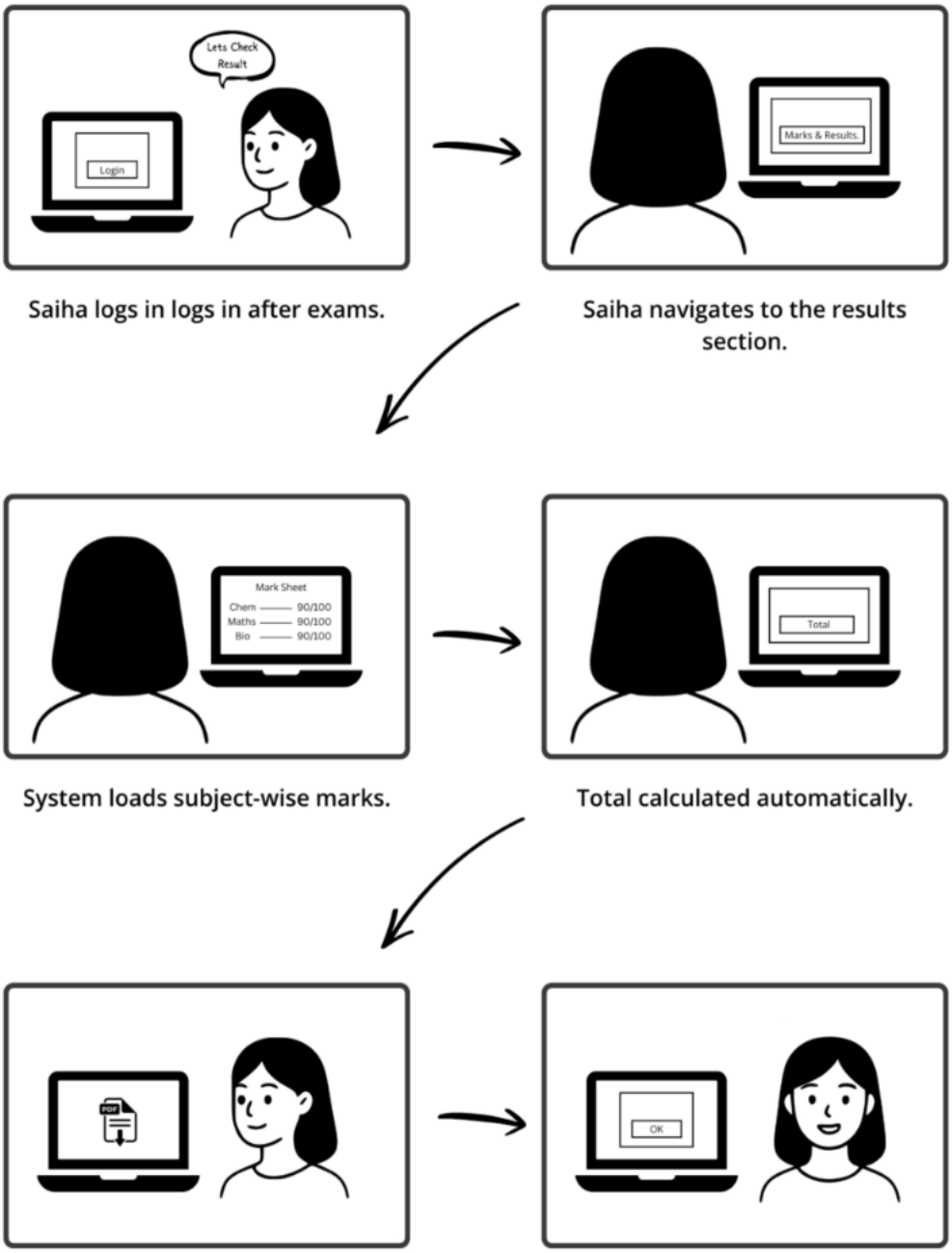



*Figure 1 Use Case Diagram*

## 3.8 Storyboards



# SCENARIO 2: STUDENT VIEWS EXAM RESULTS




Saiha logs in logs in after exams.

Saiha navigates to the results section.

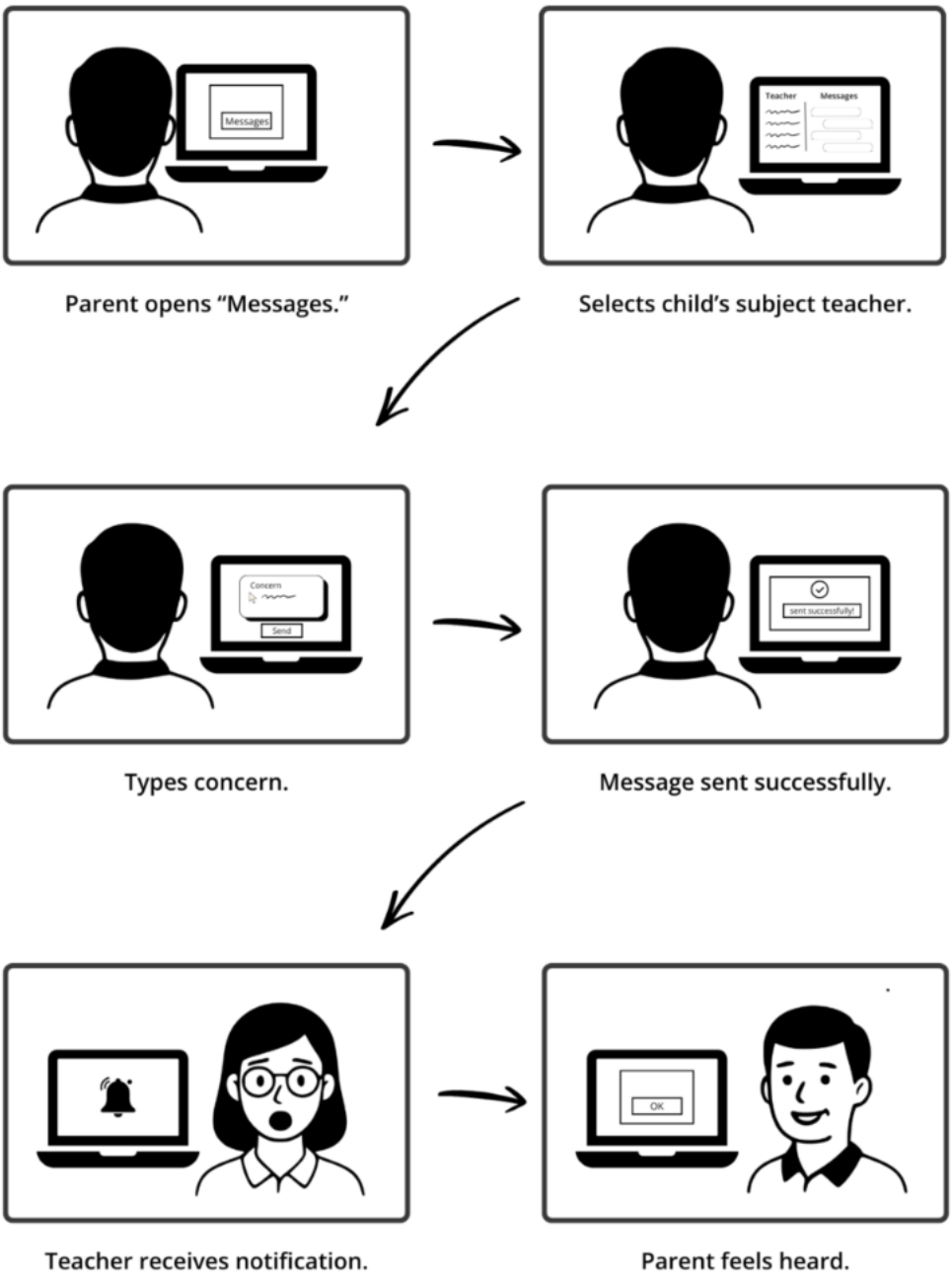
System loads subject-wise marks.

Total calculated automatically.

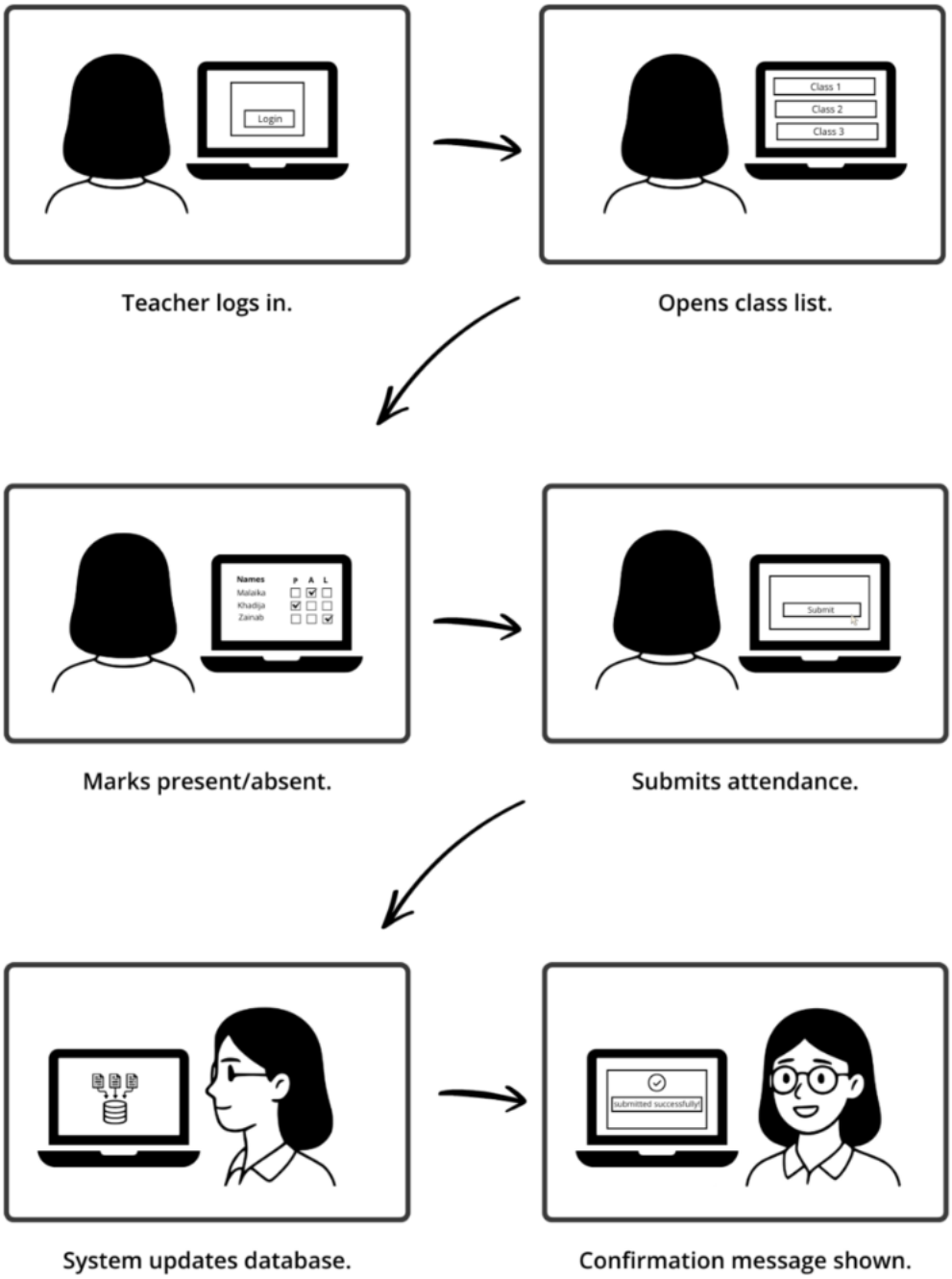
Saiha downloads result PDF.

Saiha feels satisfied and relieved.

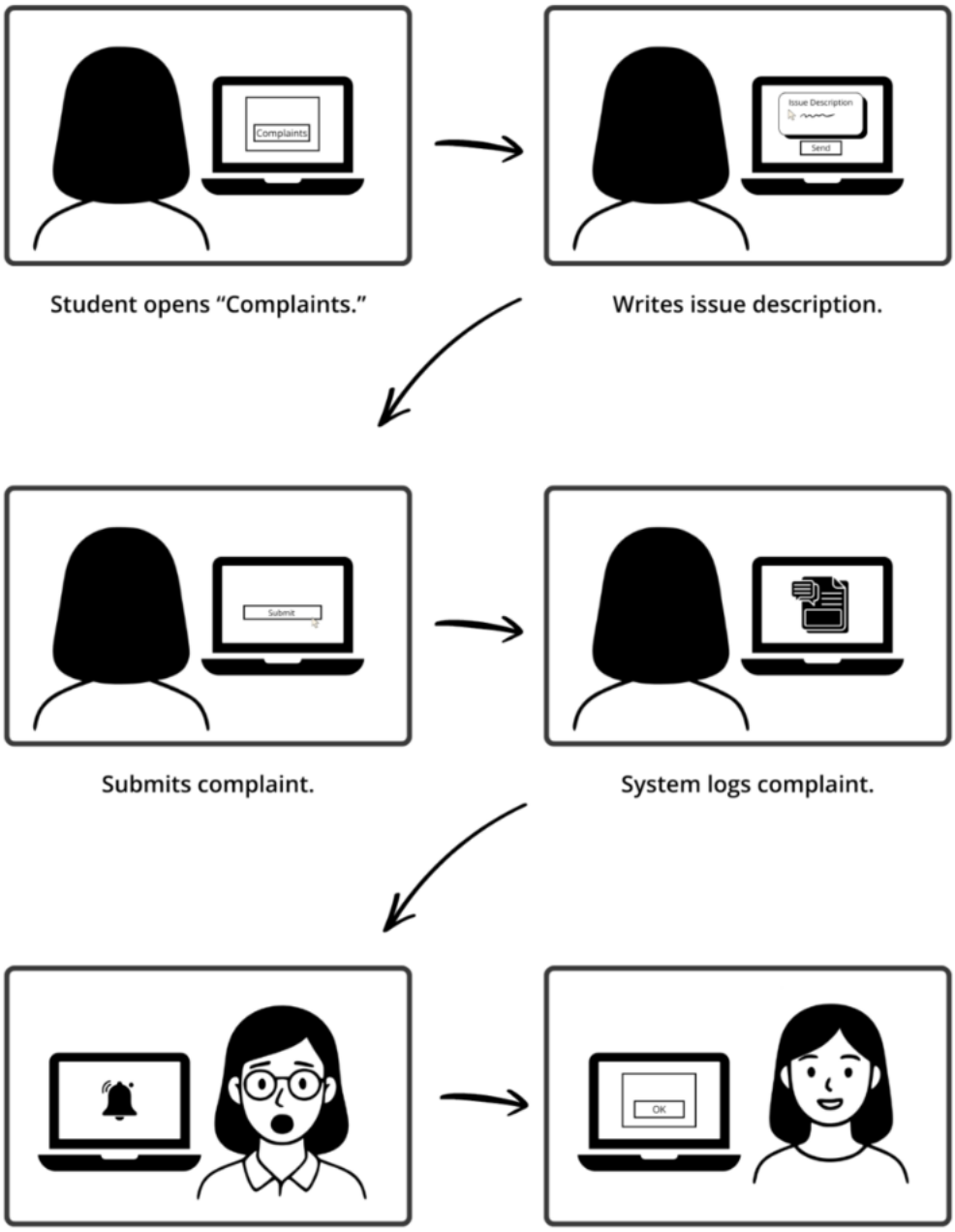
# SCENARIO 3:
# PARENT SENDS MESSAGE TO TEACHER



# SCENARIO 4: TEACHER MARKS ATTENDANCE



# SCENARIO 5: STUDENT SUBMITS COMPLAINT




Student opens "Complaints."

Writes issue description.

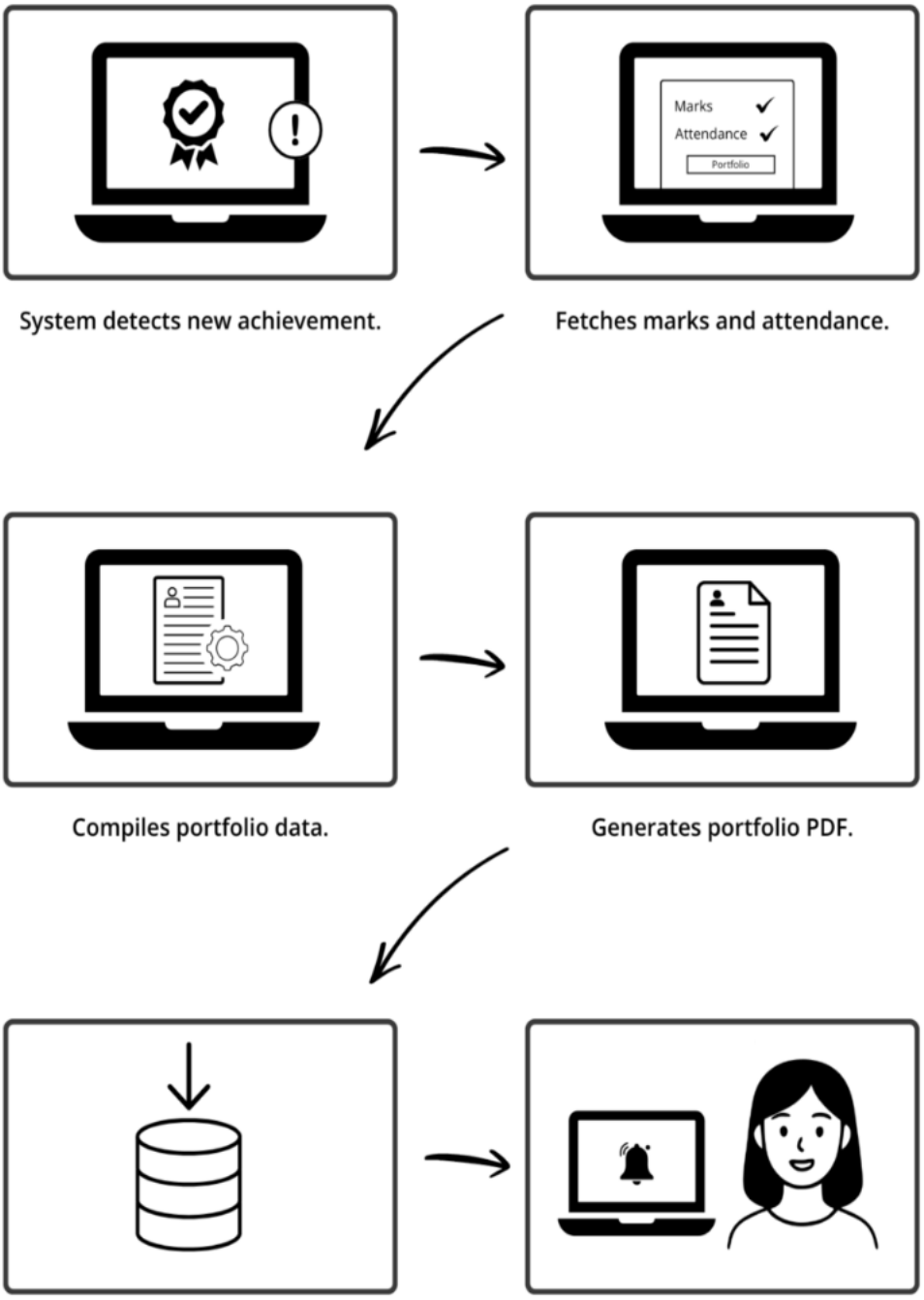
Submits complaint.

System logs complaint.

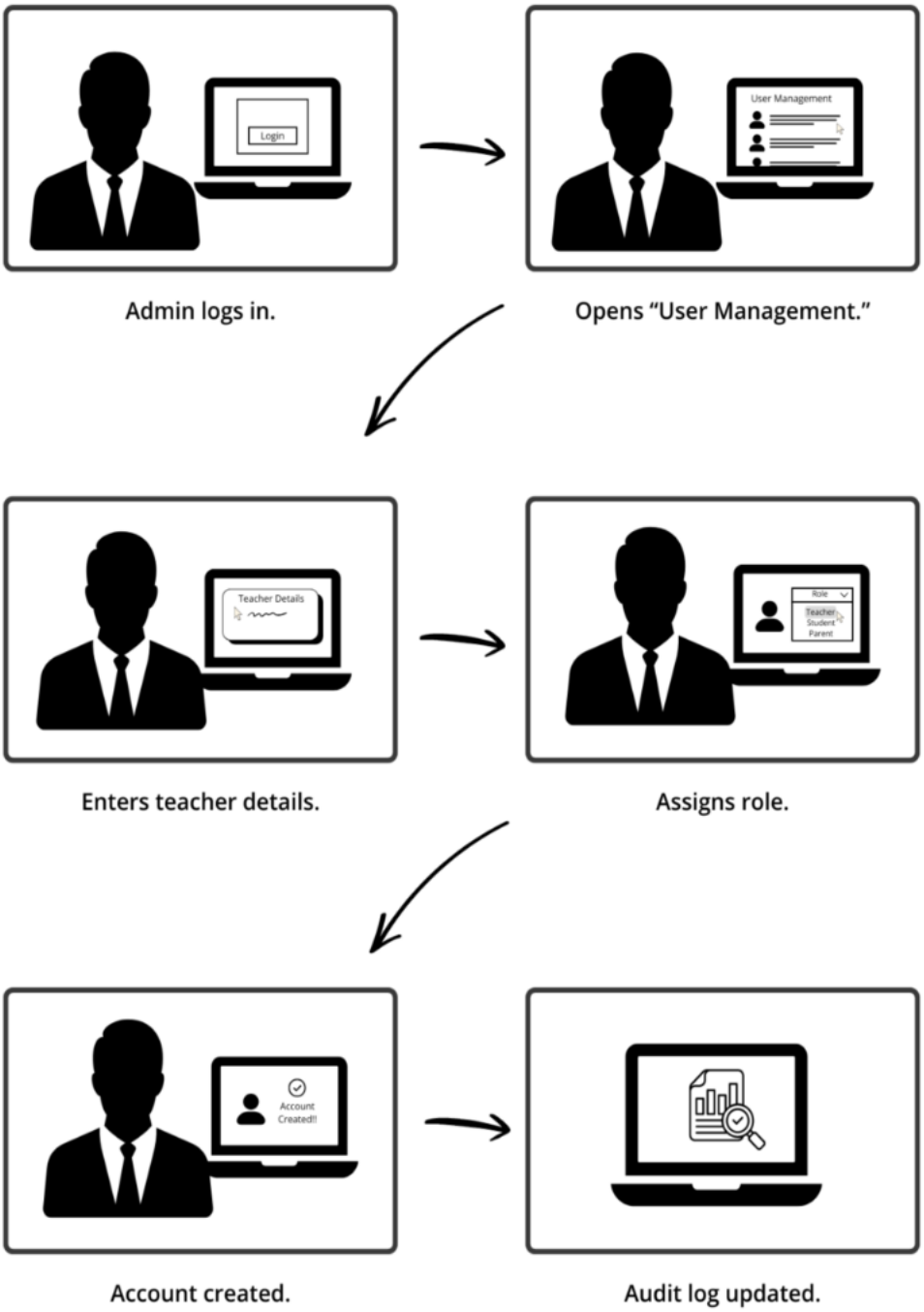
Admin/Teacher notified.

Student feels relieved.

# SCENARIO 6:
# SYSTEM GENERATES STUDENT PORTFOLIO



# SCENARIO 7: ADMIN MANAGES USERS



## 3.9 Summary

This Software Requirements Specification (SRS) documents the complete requirements engineering process carried out for the development of the **Web-Based School Management System (SMS).** The requirements were gathered and refined using multiple techniques, including stakeholder discussions, informal interviews, proposal analysis, use-case modeling, and storyboard-based workflow visualization. These techniques helped ensure that the identified requirements accurately reflect real-world academic and administrative processes within a school environment.

**Functional requirements** were systematically defined for all core modules, including **user authentication, attendance management, marks entry, student portfolio generation, risk monitoring, complaint handling, fee status management, messaging, and notifications.** Each requirement was documented with clear descriptions, business rules, dependencies, priorities, and error-handling conditions to ensure clarity, traceability, and testability.

**Non-functional requirements** were specified to address system quality attributes such as **reliability, usability, performance, and security.** These requirements were made measurable and verifiable to support validation during system testing. **External interface requirements** further defined how the system interacts with **users, software components, hardware platforms, and communication protocols.**

**Use case analysis and storyboarding** were employed to validate functional requirements from the perspective of different user roles and to visualize system behavior in practical scenarios. Overall, this **requirements engineering process** ensures that the proposed system aligns with project objectives, stakeholder expectations, and academic standards, providing a strong foundation for system design, implementation, and evaluation.

# Chapter No 4
# Software Design Specification

## 4.1 System Design

### Product Perspective

The Web-Based Unified School Management System is designed as a centralized, web-based application that integrates academic and administrative operations of an educational institution. The system operates as an independent platform accessible through standard web browsers and does not require client-side installation. It interacts with a centralized database server for storing and retrieving academic and administrative data and may integrate with external services such as email or notification systems for communication purposes.

The system follows a layered architecture where the presentation layer handles user interaction, the application layer manages business logic, and the data layer is responsible for persistent storage. This separation ensures scalability, maintainability, and ease of future enhancements. All system components communicate through well-defined interfaces to ensure reliable and secure data exchange.

### Design Constraints

Several constraints influenced the system design. Performance constraints require the system to handle multiple concurrent users with minimal response time. Security constraints demand secure authentication, authorization, and data protection mechanisms to safeguard sensitive academic and administrative information. Compatibility constraints ensure that the system functions across commonly used web browsers and devices. Additionally, the design must be scalable to accommodate increasing numbers of users and records without major architectural changes.

## 4.2 Design Considerations

### Assumptions and Dependencies:

It is assumed that users will have access to a stable internet connection and modern web browsers. The system depends on the availability of a reliable database management system and web server for proper functionality. It is also assumed that institutional data provided to the system is accurate and maintained regularly. The design assumes adherence to standard web technologies and frameworks to ensure smooth development and deployment.

### Limitations:

The system is dependent on internet availability and may not function in offline environments. Performance may be affected under extremely high loads without additional server resources. Integration with third-party systems is limited to predefined interfaces and may require further customization for advanced interoperability. Additionally, the system's functionality is constrained by the scope defined in the Software Requirements Specification (SRS).

**Risks:**

Potential risks include data security breaches, system performance degradation, and improper data handling by users. These risks are mitigated through secure authentication mechanisms, role-based access control, regular data backups, and optimized database design. Proper validation and error-handling mechanisms are incorporated to minimize system failures and ensure reliable operation.

## 4.3 Requirements Traceability Matrix

*Table 47 Requirements Traceability Matrix*

| Requirement ID | Scope | Requirement Description | Design Specification |
|---|---|---|---|
| R1 | Authentication & Security | The system shall provide secure authentication and role-based access for Admin, Teacher, Student, and Parent users. | Authentication Controller, Role-Based Authorization Middleware, ASP.NET Core Identity, User & Role Tables |
| R2 | Administration | The system shall allow administrators to manage users, classes, subjects, and teacher–subject mappings. | Admin Management Module, User Management Service, ClassController, SubjectController, EF Core Repositories |
| R3 | Attendance | The system shall allow teachers to mark and manage student attendance digitally. | Attendance Management Module, AttendanceController, AttendanceService, Attendance Table |
| R4 | Marks | The system shall allow teachers to enter, update, and manage student exam marks. | Marks Management Module, MarksController, ExamService, Marks Table |
| R5 | Dashboard | The system shall provide students and parents with real-time access to attendance and academic results. | Student Dashboard, Parent Dashboard, Academic Records API, React UI Components |
| R6 | Portfolio | The system shall generate a centralized digital student portfolio | Student Portfolio Module, PortfolioService, PortfolioController, |

| | | | |
|---|---|---|---|
| | | including attendance, grades, achievements, and certificates. | Portfolio Aggregation Logic |
| R7 | Communication | The system shall enable secure parent–teacher communication through private messaging. | Communication Module, MessagingController, MessageService, Messages Table |
| R8 | Noticeboard | The system shall provide a digital noticeboard for school-wide announcements. | Noticeboard Module, AnnouncementController, Notification Service |
| R9 | Announcements | The system shall allow administrators and teachers to post announcements and notifications. | Admin Panel, Teacher Panel, Announcement Management Service |
| R10 | Complaints | The system shall allow students to submit complaints or behavior-related concerns. | Complaint Management Module, ComplaintController, Complaint Submission UI |
| R11 | Complaints | The system shall allow teachers to add remarks and administrators to resolve and track complaints. | Complaint Workflow Engine, ComplaintStatus Enum, Complaint Resolution Service |
| R12 | Audit & Compliance | The system shall track and maintain complaint and behavior history for audit and transparency. | Complaint History Table, Audit Logs, EF Core Change Tracking |
| R13 | Fees | The system shall display fee status and reminders to students and parents. | Fee Management Module, FeeController, FeeStatus Table |
| R14 | Risk Analysis | The system shall analyze attendance and academic performance to highlight student risk indicators. | Risk Analysis Service, Performance Evaluation Logic, Dashboard Alerts |
| R15 | Notifications | The system shall provide centralized dashboard notifications for users based on their roles. | Notification Engine, Role-Based Notification Service, React Notification Components |
| R16 | Data Management | The system shall maintain secure storage and retrieval of | SQL Server Database, EF Core ORM, Data Access Layer |

| | | | |
|---|---|---|---|
| | | academic and administrative records. | |
| R17 | UI/UX | The system shall support responsive web access across desktop and mobile devices. | React Frontend, Responsive UI Design, Tailwind CSS / Bootstrap |
| R18 | Security & Privacy | The system shall ensure data privacy, integrity, and controlled access to sensitive information. | Authorization Policies, HTTPS, JWT Tokens, Secure API Endpoints |

# 4.4 Design Models

The system design is described using standard UML design models to clearly represent its structure and behavior under an object-oriented development approach.

## 1. Design Class Diagram (DCD)

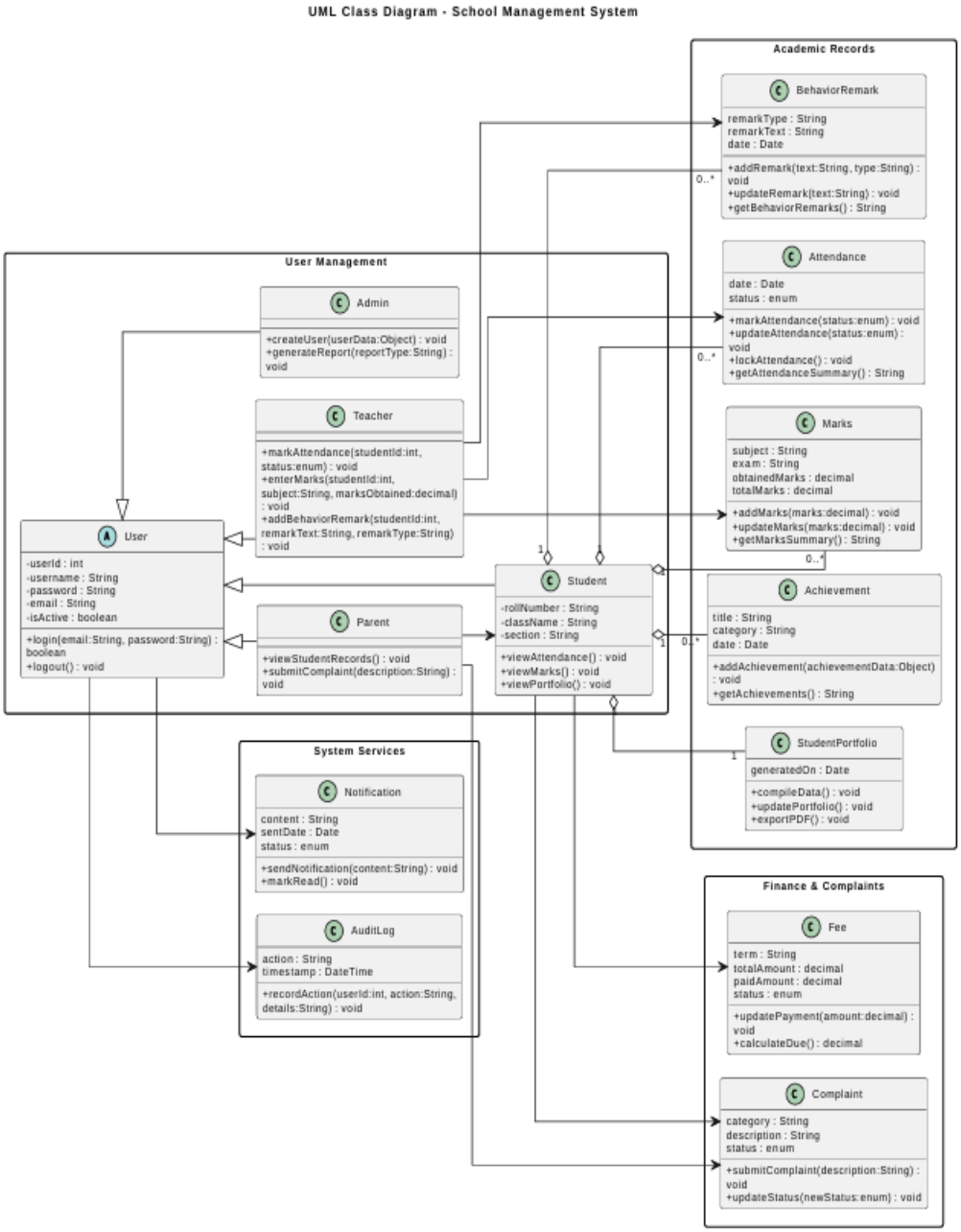



*Figure 2 Class Diagram*

## 2. Interaction Diagram (sequence)

**User Flow login – Sequence Diagram**

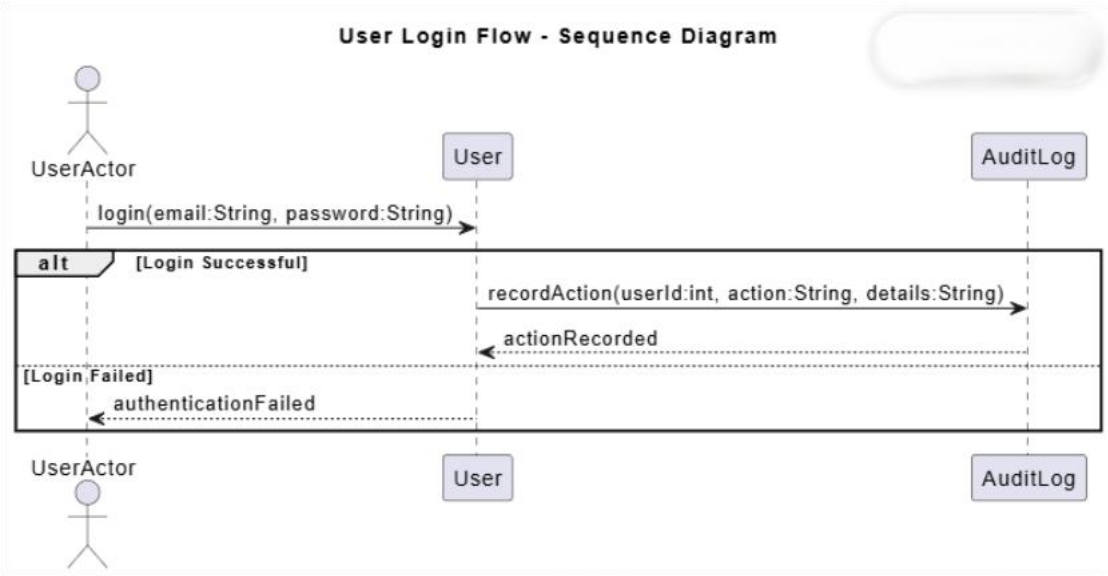



*Figure 3 Sequence Diagram 1- User Flow Login*

**Fig explanation:** This figure illustrates the user authentication process, where login credentials are validated and successful login activities are recorded through the audit logging service. It ensures secure access and accountability within the system.

**Teacher Academic Management Flow – Sequence Diagram**

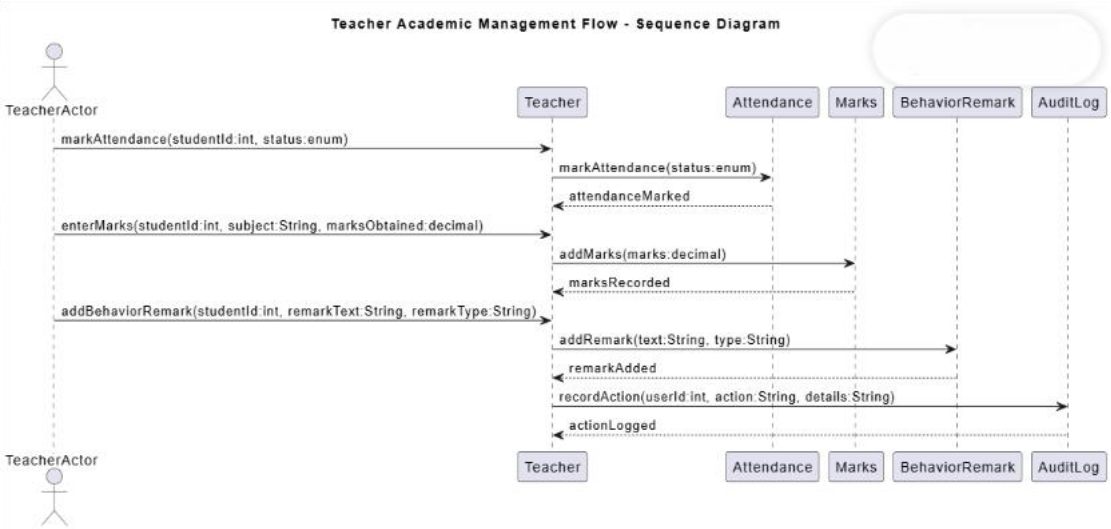



*Figure 4 Sequence Diagram 2- Teacher Academic Management Flow*

**Fig explanation:** This figure shows how a teacher manages academic activities, including marking attendance, entering marks, and adding behavior remarks. Each action is logged to maintain data integrity and accountability.

**Student Portfoilio Generation Flow – Sequence Diagram**

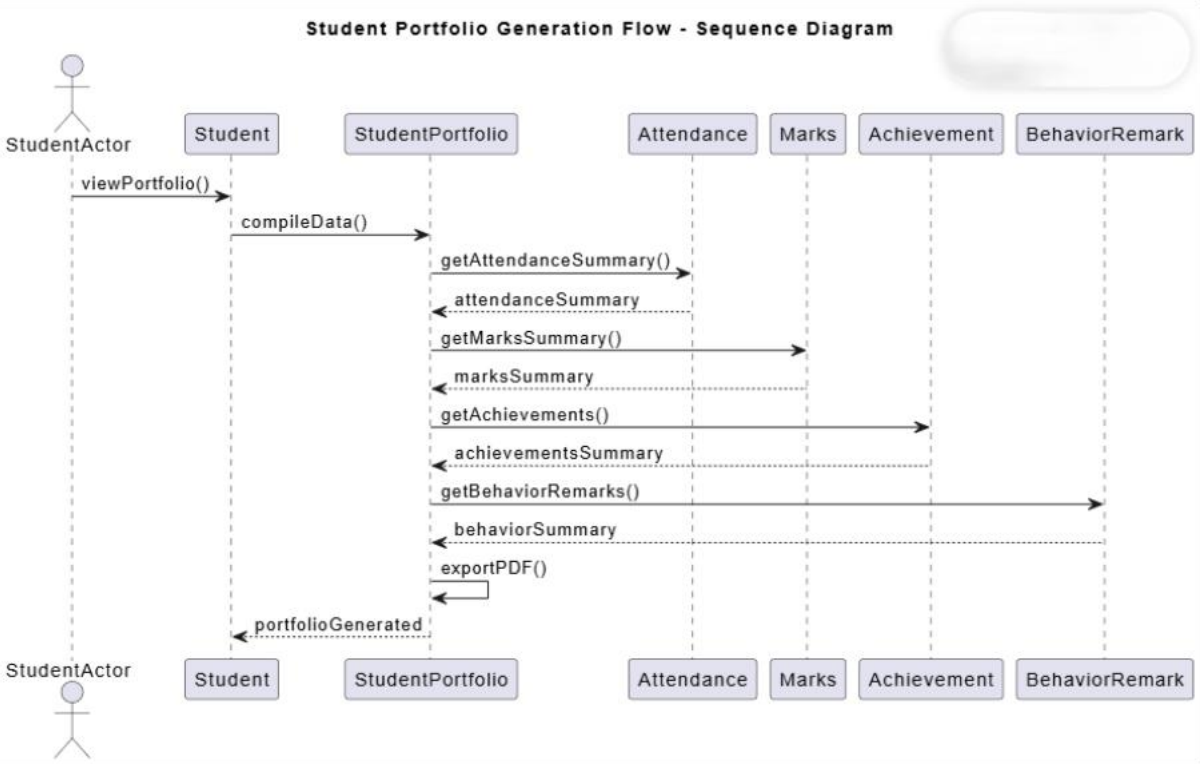



*Figure 5 Sequence Diagram 3- Student Portfolio Generation Flow*

**Fig explanation:** This figure depicts the process of generating a student portfolio by aggregating attendance, marks, achievements, and behavior records. The compiled data is then exported as a complete academic performance report.

## 3. State Transition Diagram

**Attendance Record State Diagram**

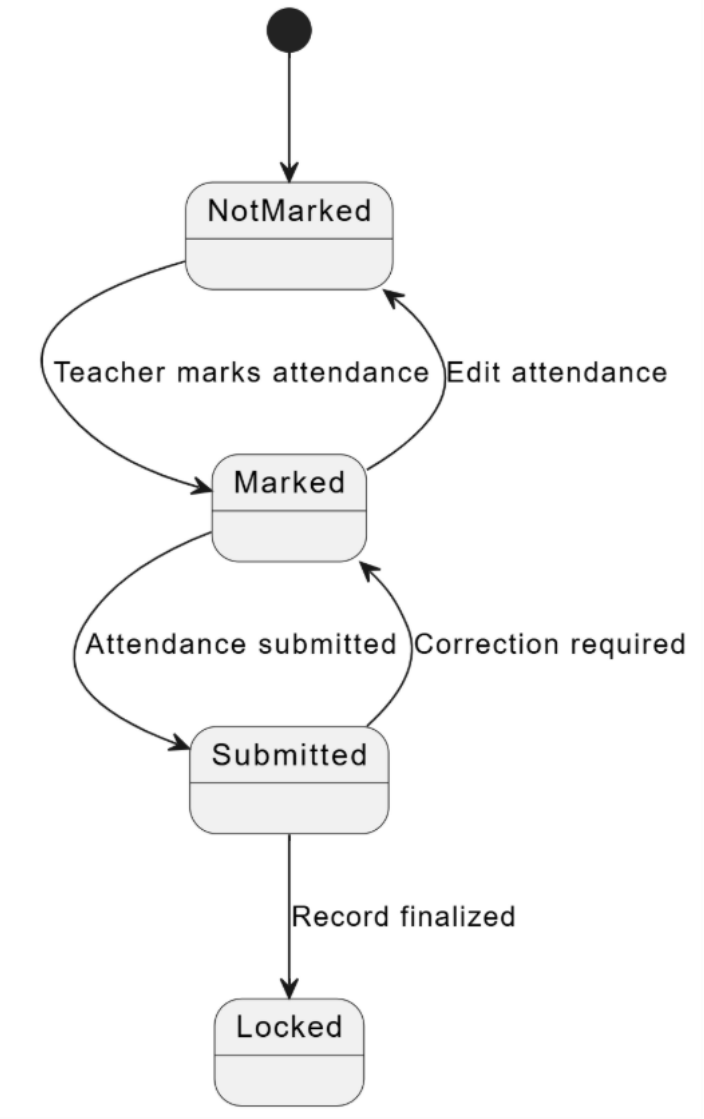



*Figure 6 State Diagram 1- Attendance Record*

**Fig explanation:** This state diagram models the attendance recording process. Attendance starts unmarked, is entered by the teacher, submitted for confirmation, and finally locked to prevent unauthorized changes.

**Complaint Management State Diagram**

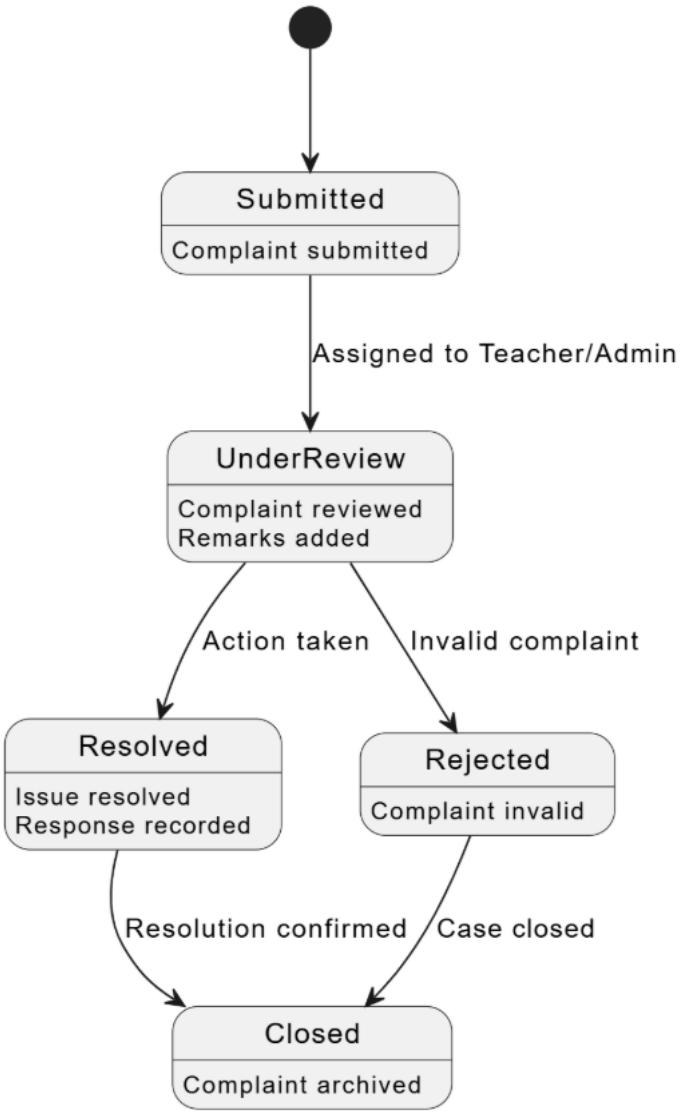



*Figure 7 State Diagram 2- Complain Mangement*

**Fig explanation:** This diagram illustrates the complaint handling workflow. A complaint is submitted by a student or parent, reviewed by staff, resolved or rejected, and finally closed after confirmation.

**Student Portfolio State Diagram**

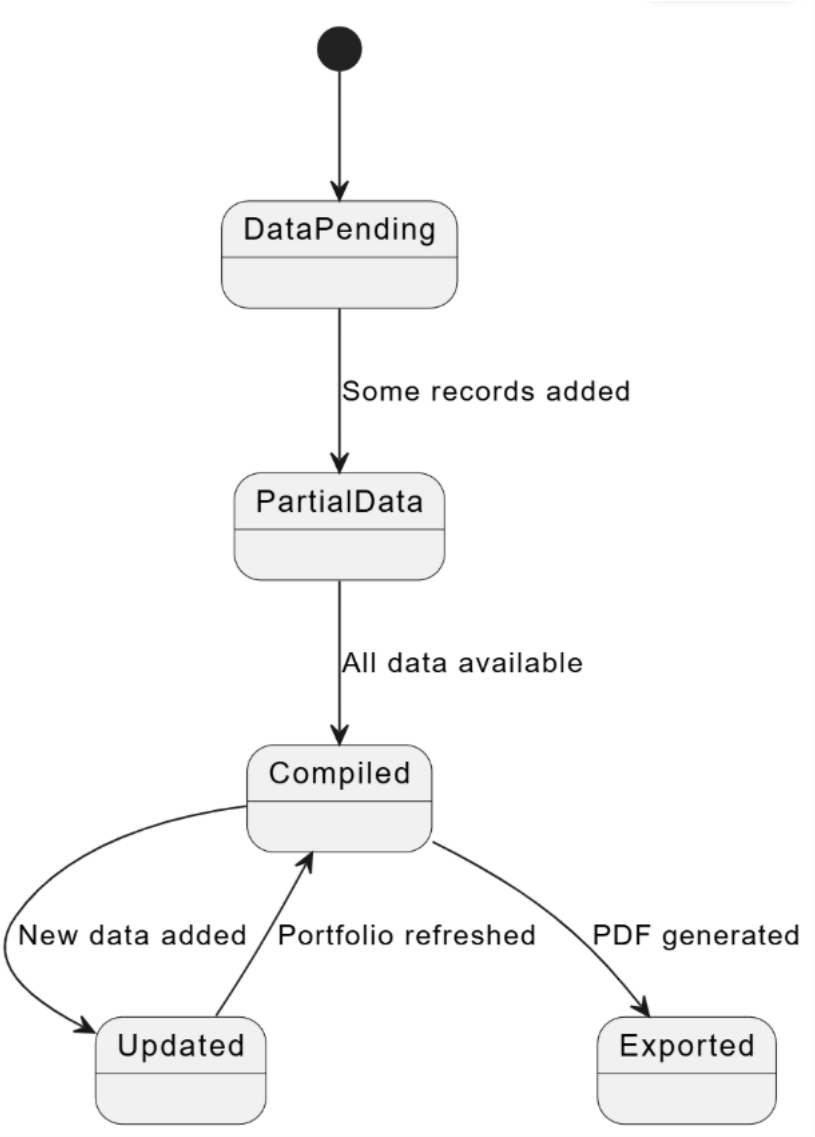



*Figure 8 State Diagram 3- Student Portfolio*

**Fig explanation:** This diagram represents the automated student portfolio generation process. The system collects academic data, compiles it into a portfolio, updates it when new data arrives, and allows export in PDF format.

**Fee payment State Diagram**

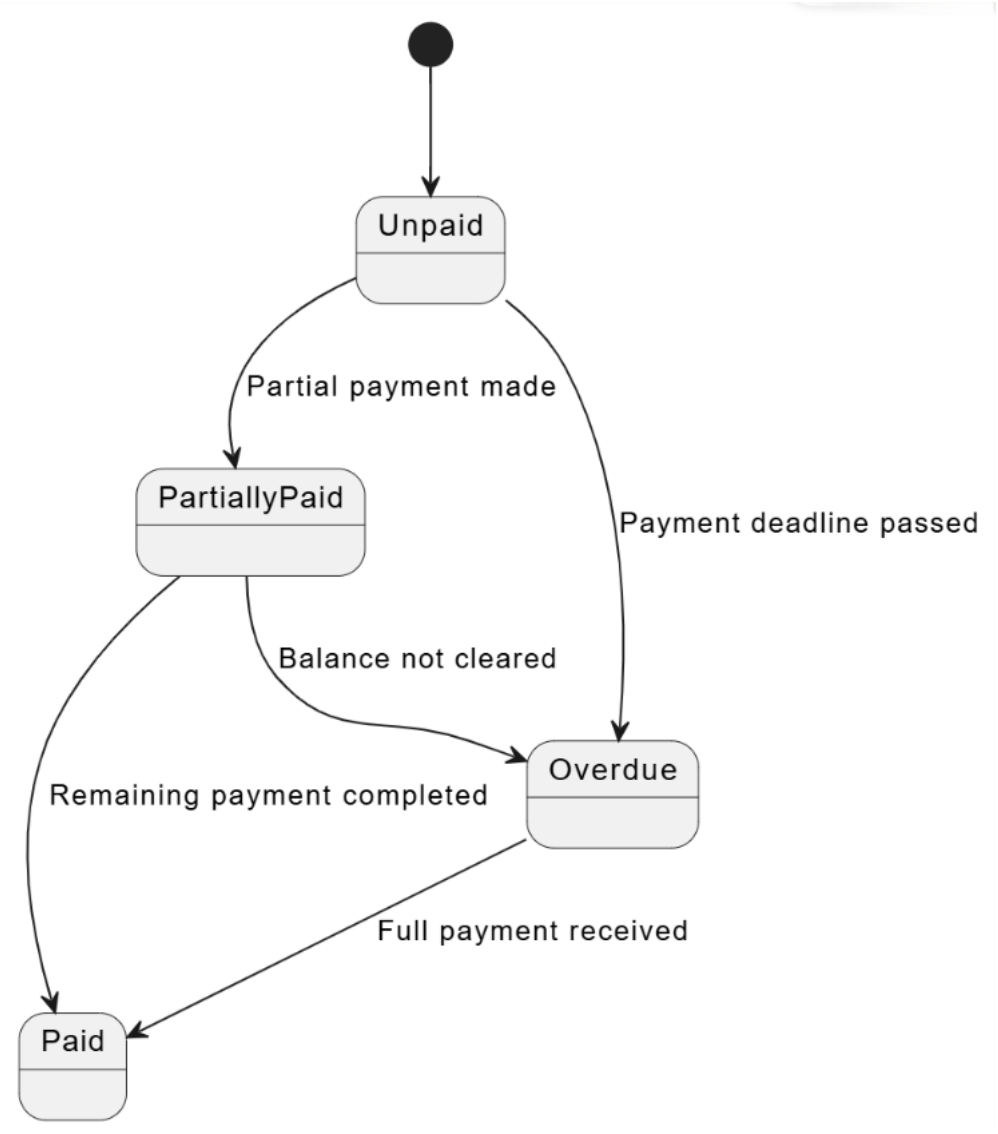



*Figure 9 State Diagram 4- fee Payment*

**Fig explanation:** This diagram shows the fee payment lifecycle of a student. It tracks payment status from unpaid to partially paid, overdue, and fully paid.

**User Account State Diagram**

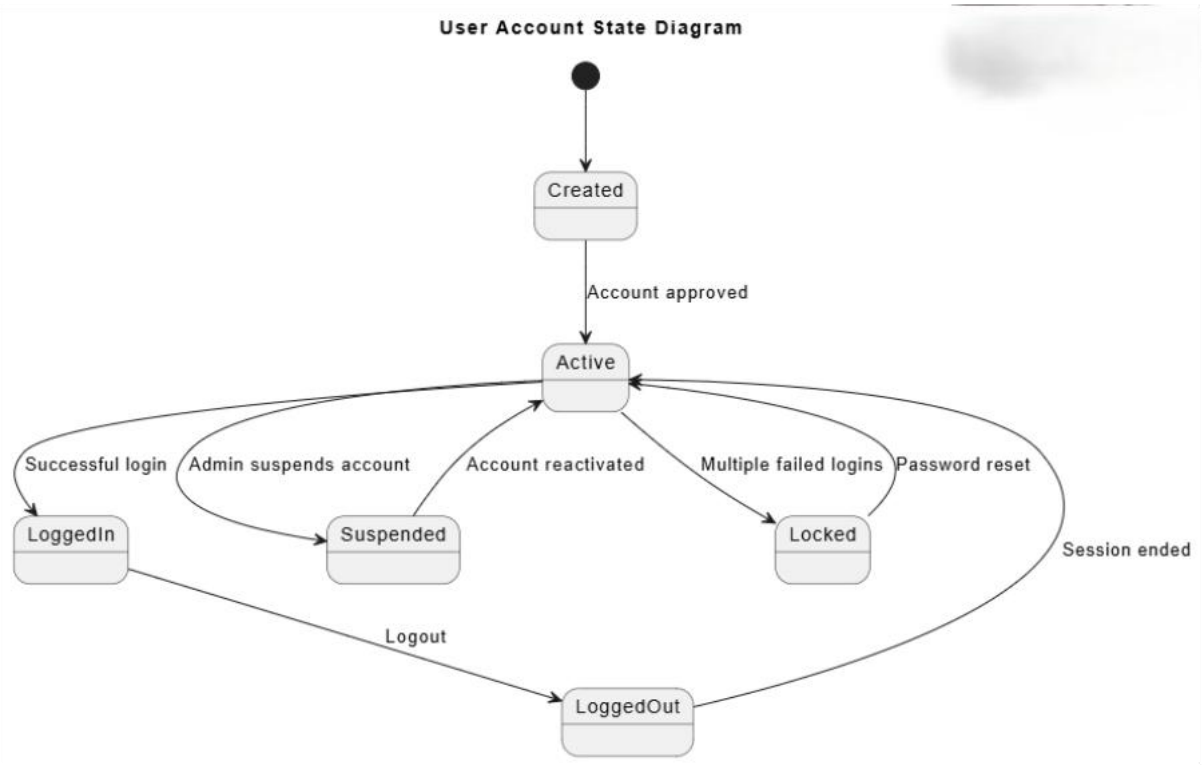



*Figure 10 State Diagram 5- User account*

**Fig explanation:** This state diagram represents the lifecycle of a user account in the system. It shows transitions from account creation to activation, login/logout behavior, and administrative actions such as suspension, locking, and reactivation.

# 4.5 Architectural Design

***Component Structure and Inter-Module Relationships:***

The system architecture is designed using a layered and component-based approach, following the MVC and client–server architectural patterns. The architecture separates concerns across presentation, application, and data tiers, enabling scalability, maintainability, and secure role-based access. Each layer communicates with the adjacent layer through well-defined interfaces and APIs to ensure controlled data flow and modular development.

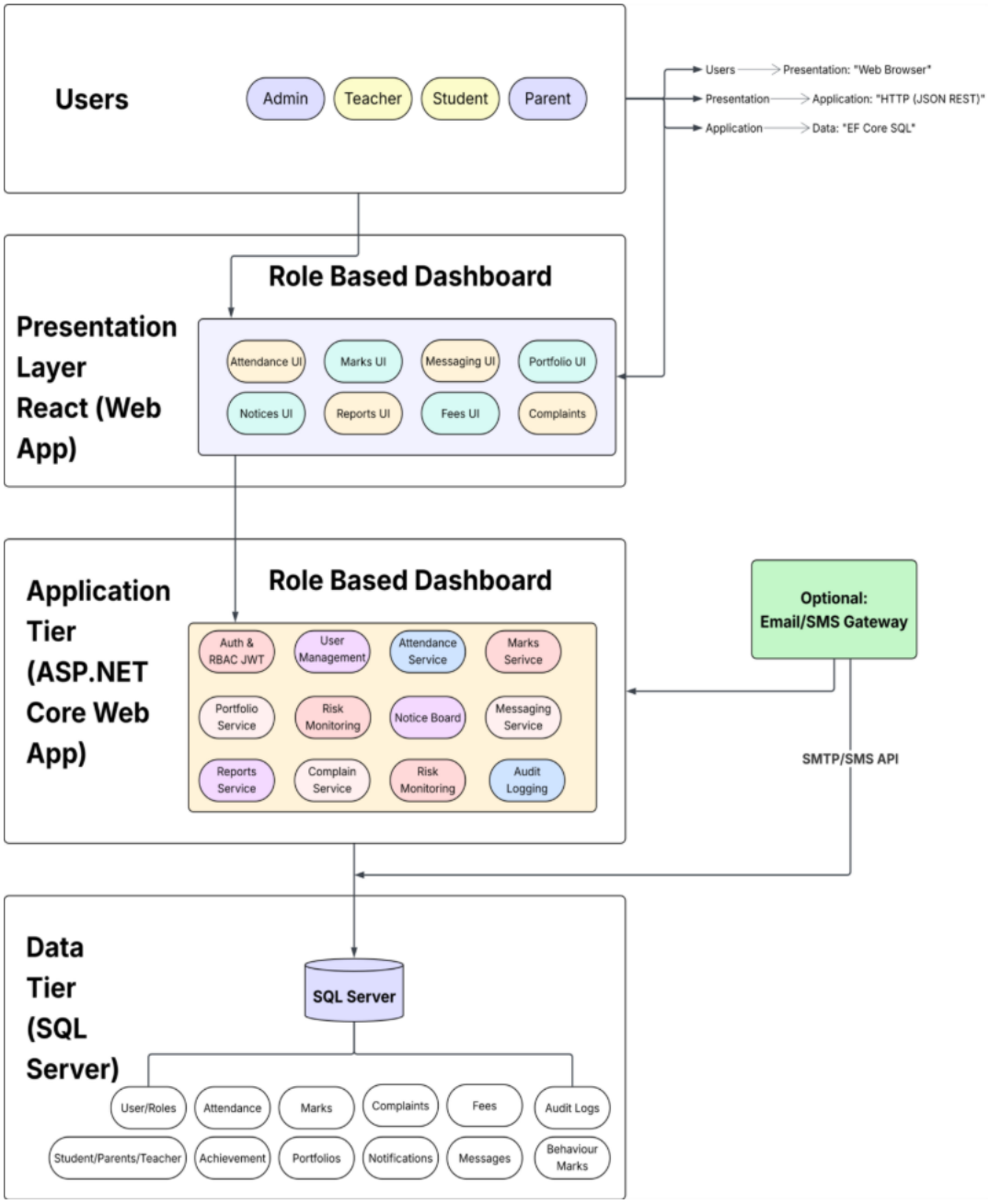



*Figure 11 System Architecture Diagram*

**Fig explanation:** This figure illustrates the system interaction flow in which user requests originate from the client-side interface and are processed by application services through the presentation layer. Business logic execution and data persistence are handled via the data access layer. Role-Based Access Control (RBAC) and authentication mechanisms are enforced prior to any operation, ensuring secure and authorized access to system functionalities.

### *Mapping Components to Architecture Layers*

| Architecture Layer | MVC Role | Component / Module | Description |
|---|---|---|---|
| Client Layer | View | Web UI / Browser | Provides user interface for Admin, Teacher, Student, and Parent to view dashboards, forms, and reports |
| Application Layer | Controller | Controllers (Authentication, Attendance, Marks, Complaint, Fee) | Handles user requests, input validation, and coordination between the presentation layer and business logic |
| Business Layer | Model | User Management | Manages user accounts, roles, authentication, and role-based access control |
| Business Layer | Model | Academic Users | Represents core system entities including Student, Teacher, Parent, and Admin along with their operations |
| Business Layer | Model | Academic Records | Handles attendance, marks, behavior remarks, achievements, and student portfolios |
| Business Layer | Model | Complaints & Fees | Manages complaint registration, tracking, resolution, and fee payment processing |
| Service Layer | — | System Services | Provides cross-cutting services such as notifications, risk monitoring, and audit logging |

| | | | |
|---|---|---|---|
| Data Layer | — | Relational Database | Stores all persistent system data including users, academic records, complaints, fees, logs, and reports |

# 1. UML Component diagram

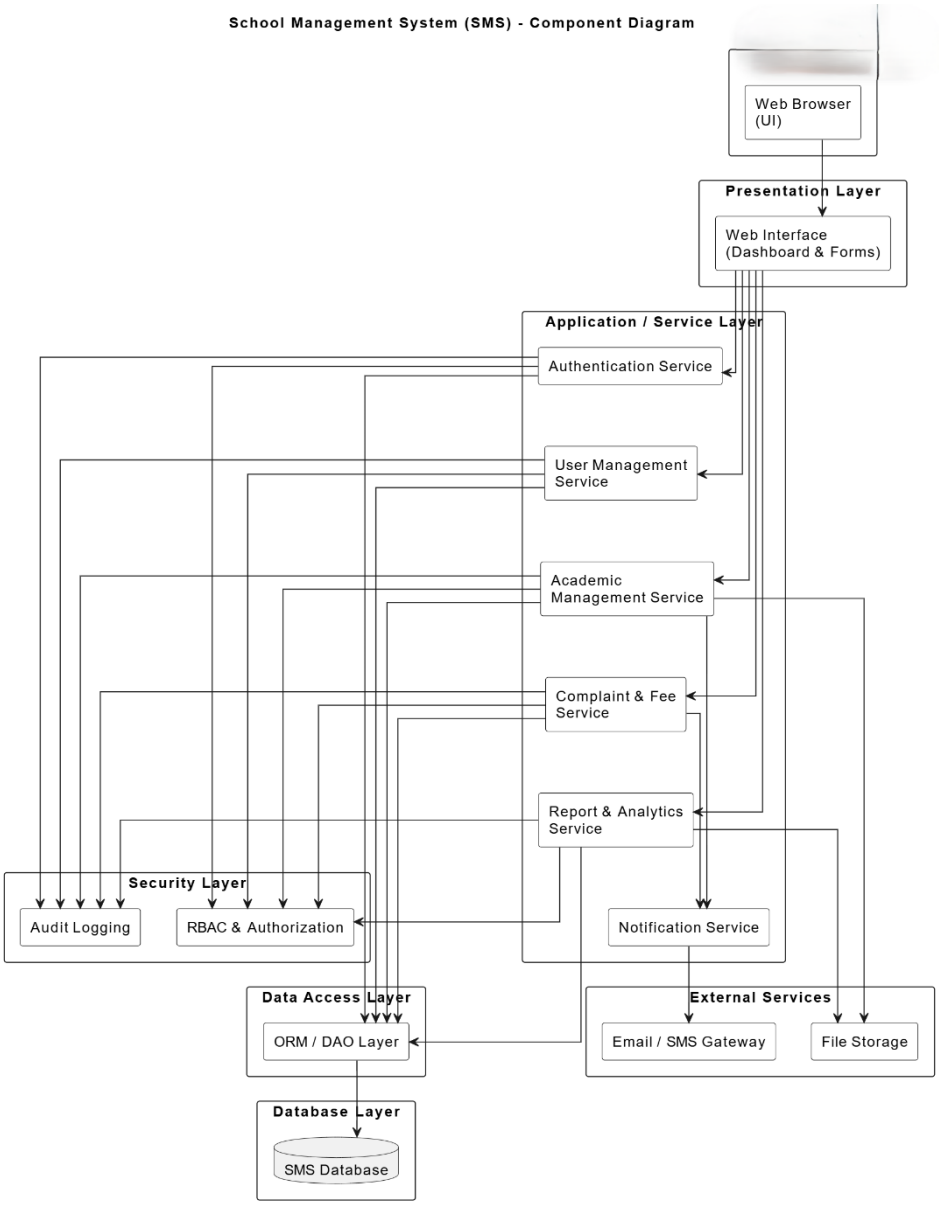



*Figure 12 UML Component Diagram*

**Figure explanation:** This Figure illustrates the UML Component Diagram of the School Management System, representing a layered MVC-based client–server architecture with integrated security, data access, and external services.

## 4.6 Data Design

***Information Domain Transformation:***

The information domain defined in the Software Requirements Specification (SRS) is transformed into structured data entities using a strictly **Object-Oriented (OO) design approach**. Each functional requirement is mapped to one or more domain objects that encapsulate both **data (attributes)** and **behavior (methods)**. This mapping ensures full traceability between the SRS and the Software Design Specification (SDS).

The system strictly supports the four user roles defined in the SRS: **Admin, Teacher, Student, and Parent**. No additional actors or roles are introduced at the design level. Role-specific behavior is enforced through object methods and access control rather than dynamic role assignment.

Persistent data are stored in a **relational database**, while temporary data such as user sessions and notifications are handled at the application layer. Business rules and validations are implemented through object methods, ensuring separation between data persistence and application logic.

***Data Storage and Organization:***

- **Database Type:** Relational Database (MySQL / PostgreSQL)
- **Design Paradigm:** Object-Oriented domain model mapped to relational tables
- **Normalization Level:** Third Normal Form (3NF) to reduce redundancy
- **Security Measures:**
    a. Encrypted password storage
    b. Role-based access control
    c. Audit logging of critical actions
- **Data Processing:**
    a. CRUD operations performed via service-layer object methods
    b. Referential integrity enforced using primary and foreign keys

***Major System Entities:***

The major system entities derived directly from the SRS are:

- User (Admin, Teacher, Student, Parent)
- Student

- Parent
- Teacher
- Admin
- Attendance
- Marks
- Achievement
- BehaviorRemark
- Complaint
- Fee
- Notification
- Message
- StudentPortfolio
- AuditLog

## 1. Data Dictionary

| **Terminology** | **Description** |
|---|---|
| Achievement | Object representing student achievements. **Attributes:** achievementId:int (PK), studentId:int (FK → Student), title:String, category:String, date:Date, filePath:String. Used to store awards, certificates, and extracurricular accomplishments. |
| Achievement.date | Date of achievement. **Type:** Date |
| Achievement.filePath | File path for attachment/document. **Type:** String |
| Achievement.title | Name/title of achievement. **Type:** String |
| Achievement.category | Type/category of achievement (e.g., Sports, Academic). **Type:** String |
| Achievement.achievementId | Primary key identifier. **Type:** int |
| Achievement.studentId | Foreign key to Student. **Type:** int |
| Achievement.addAchievement() | Method to add a new achievement. Parameters: achievementData:Object |
| Admin | Object representing administrative user. **Attributes:** adminId:int (PK), userId:int (FK → User), firstName:String, lastName:String. |

| | |
|---|---|
| | Admin manages users, fees, notices, reports, and system settings. |
| Admin.adminId | Primary key identifier. **Type:** int |
| Admin.userId | Foreign key to User. **Type:** int |
| Admin.firstName | Admin first name. **Type:** String |
| Admin.lastName | Admin last name. **Type:** String |
| Admin.createUser(userData:Object) | Method to create a new user account. |
| Admin.generateReport(reportType:String) | Generates system reports. Parameters: reportType:String |
| Admin.updateSettings(settings:Object) | Updates system settings. Parameters: settings:Object |
| AuditLog | Tracks system actions for accountability. **Attributes:** logId:int, userId:int (FK → User), action:String, timestamp:Timestamp, details:String |
| AuditLog.logId | Primary key. **Type:** int |
| AuditLog.userId | User performing the action. **Type:** int |
| AuditLog.action | Description of action. **Type:** String |
| AuditLog.timestamp | Action timestamp. **Type:** Timestamp |
| AuditLog.details | Additional details about action. **Type:** String |
| AuditLog.recordAction(userId:int, action:String, details:String) | Logs an action in the system. |
| Attendance | Tracks student attendance. **Attributes:** attendanceId:int, studentId:int (FK), teacherId:int (FK), date:Date, status:enum {Present, Absent, Late} |
| Attendance.attendanceId | Primary key. **Type:** int |
| Attendance.studentId | Foreign key to Student. **Type:** int |
| Attendance.teacherId | Foreign key to Teacher. **Type:** int |

| | |
|---|---|
| Attendance.date | Date of attendance. **Type:** Date |
| Attendance.status | Attendance status. **Type:** enum {Present, Absent, Late} |
| Attendance.markAttendance(status:enum) | Marks attendance for a student. |
| Attendance.updateAttendance(status:enum) | Updates existing attendance record. |
| Attendance.lockAttendance() | Locks attendance to prevent further edits. |
| BehaviorRemark | Stores teacher comments about student behavior. **Attributes:** remarkId:int, studentId:int (FK), teacherId:int (FK), remarkType:String, remarkText:String, date:Date |
| BehaviorRemark.remarkId | Primary key. **Type:** int |
| BehaviorRemark.studentId | Foreign key to Student. **Type:** int |
| BehaviorRemark.teacherId | Foreign key to Teacher. **Type:** int |
| BehaviorRemark.remarkType | Type of remark (Positive/Negative). **Type:** String |
| BehaviorRemark.remarkText | Remark content. **Type:** Text |
| BehaviorRemark.date | Date of remark. **Type:** Date |
| BehaviorRemark.addRemark(text:String, type:String) | Adds a new behavior remark. |
| BehaviorRemark.updateRemark(text:String) | Updates remark content. |
| Complaint | Represents complaints submitted by students or parents. **Attributes:** complaintId:int, submittedByUserId:int (FK), assignedToUserId:int (FK), category:String, description:String, status:enum {Submitted, UnderReview, Resolved, Rejected, Closed}, dateSubmitted:Date, dateClosed:Date |
| Complaint.complaintId | Primary key. **Type:** int |
| Complaint.submittedByUserId | Foreign key to submitting User. **Type:** int |

| | |
|---|---|
| Complaint.assignedToUserId | Foreign key to assigned User (Teacher/Admin). **Type:** int |
| Complaint.category | Complaint category. **Type:** String |
| Complaint.description | Complaint details. **Type:** Text |
| Complaint.status | Current status. **Type:** enum |
| Complaint.dateSubmitted | Submission date. **Type:** Date |
| Complaint.dateClosed | Resolution date. **Type:** Date |
| Complaint.submitComplaint(description:String) | Submits a new complaint. |
| Complaint.updateStatus(newStatus:enum) | Updates complaint status. |
| Complaint.addRemarks(remarks:String) | Adds remarks to complaint record. |
| Fee | Tracks student fee payments. **Attributes:** feeId:int, studentId:int (FK), term:String, totalAmount:decimal, paidAmount:decimal, dueDate:Date, status:enum {Paid, Partial, Pending, Overdue} |
| Fee.feeId | Primary key. **Type:** int |
| Fee.studentId | Foreign key to Student. **Type:** int |
| Fee.term | Academic term/semester. **Type:** String |
| Fee.totalAmount | Total fee amount. **Type:** decimal |
| Fee.paidAmount | Amount paid. **Type:** decimal |
| Fee.dueDate | Payment due date. **Type:** Date |
| Fee.status | Payment status. **Type:** enum |
| Fee.updatePayment(amount:decimal) | Updates paid amount. |
| Fee.calculateDue() | Calculates remaining due. |
| Marks | Stores assessment/exam results. **Attributes:** marksId:int, studentId:int (FK), teacherId:int (FK), subject:String, exam:String, marksObtained:decimal, totalMarks:decimal |

| | |
|---|---|
| Marks.marksId | Primary key. **Type:** int |
| Marks.studentId | Foreign key to Student. **Type:** int |
| Marks.teacherId | Foreign key to Teacher. **Type:** int |
| Marks.subject | Subject name. **Type:** String |
| Marks.exam | Exam/assessment name. **Type:** String |
| Marks.marksObtained | Marks obtained. **Type:** decimal |
| Marks.totalMarks | Total possible marks. **Type:** decimal |
| Marks.addMarks(marks:decimal) | Adds marks for student. |
| Marks.updateMarks(marks:decimal) | Updates existing marks. |
| Message | Internal messaging. **Attributes:** messageId:int, senderUserId:int (FK), receiverUserId:int (FK), content:String, sentAt:DateTime |
| Message.messageId | Primary key. **Type:** int |
| Message.senderUserId | User sending message. **Type:** int |
| Message.receiverUserId | User receiving message. **Type:** int |
| Message.content | Message content. **Type:** String |
| Message.sentAt | Timestamp sent. **Type:** DateTime |
| Message.sendMessage(content:String, receiverId:int) | Sends a new message. |
| Message.readMessage() | Marks message as read. |
| Notification | Stores system notifications. **Attributes:** notificationId:int, userId:int (FK), type:String, content:String, dateSent:Date, status:enum {Unread, Read, Archived} |
| Notification.notificationId | Primary key. **Type:** int |
| Notification.userId | Recipient user. **Type:** int |

| | |
|---|---|
| Notification.type | Notification type (e.g., Info, Warning). **Type:** String |
| Notification.content | Message content. **Type:** Text |
| Notification.dateSent | Date sent. **Type:** Date |
| Notification.status | Read status. **Type:** enum |
| Notification.sendNotification(content:String) | Sends notification to user. |
| Notification.markRead() | Marks notification as read. |
| Parent | Represents a parent user. **Attributes:** parentId:int, userId:int (FK), studentId:int (FK), relation:String |
| Parent.parentId | Primary key. **Type:** int |
| Parent.userId | Foreign key to User. **Type:** int |
| Parent.studentId | Foreign key to Student. **Type:** int |
| Parent.relation | Relation to student (Father/Mother/Guardian). **Type:** String |
| Parent.viewStudentRecords() | Accesses linked student's academic records. |
| Portfolio | Compiled academic record. **Attributes:** portfolioId:int, studentId:int (FK), attendanceSummary:String, marksSummary:String, achievementsSummary:String, behaviorSummary:String, generatedOn:Date |
| Portfolio.portfolioId | Primary key. **Type:** int |
| Portfolio.studentId | Foreign key to Student. **Type:** int |
| Portfolio.attendanceSummary | Text summary of attendance. **Type:** String |
| Portfolio.marksSummary | Text summary of marks. **Type:** String |
| Portfolio.achievementsSummary | Text summary of achievements. **Type:** String |
| Portfolio.behaviorSummary | Text summary of behavior remarks. **Type:** String |

| | |
|---|---|
| Portfolio.generatedOn | Date generated. **Type:** Date |
| Portfolio.compileData() | Compiles portfolio data from different modules. |
| Portfolio.updatePortfolio() | Updates existing portfolio record. |
| Portfolio.exportPDF() | Exports portfolio as PDF. |
| Report | System-generated reports. **Attributes:** reportId:int, type:String, generatedOn:Date, filePath:String, adminId:int (FK) |
| Report.reportId | Primary key. **Type:** int |
| Report.type | Type of report (Attendance, Marks, Portfolio, Risk). **Type:** String |
| Report.generatedOn | Date generated. **Type:** Date |
| Report.filePath | File location. **Type:** String |
| Report.adminId | Generated by Admin. **Type:** int |
| Report.generateReport(type:String) | Creates a report file. |
| RiskMonitoring | Monitors student risk. **Attributes:** riskId:int, studentId:int (FK), riskLevel:String {Low, Medium, High}, monitoredOn:Date, notes:String |
| RiskMonitoring.riskId | Primary key. **Type:** int |
| RiskMonitoring.studentId | Foreign key to Student. **Type:** int |
| RiskMonitoring.riskLevel | Risk rating. **Type:** String |
| RiskMonitoring.monitoredOn | Date of assessment. **Type:** Date |
| RiskMonitoring.notes | Additional info. **Type:** String |
| RiskMonitoring.assessRisk() | Evaluates current risk level. |
| RiskMonitoring.updateRisk(level:String, notes:String) | Updates risk details. |

| | |
|---|---|
| Student | Represents student. **Attributes:** studentId:int, userId:int (FK), firstName:String, lastName:String, class:String, section:String, rollNumber:String |
| Student.studentId | Primary key. **Type:** int |
| Student.userId | Foreign key to User. **Type:** int |
| Student.firstName | First name. **Type:** String |
| Student.lastName | Last name. **Type:** String |
| Student.class | Class/grade. **Type:** String |
| Student.section | Section/group. **Type:** String |
| Student.rollNumber | Roll number. **Type:** String |
| Student.viewAttendance() | Views attendance records. |
| Student.viewMarks() | Views marks. |
| Student.viewPortfolio() | Accesses compiled portfolio. |
| Student.submitComplaint(description:String) | Submits complaint. |
| Teacher | Represents teacher. **Attributes:** teacherId:int, userId:int (FK), firstName:String, lastName:String, assignedSubjects:String |
| Teacher.teacherId | Primary key. **Type:** int |
| Teacher.userId | Foreign key to User. **Type:** int |
| Teacher.firstName | First name. **Type:** String |
| Teacher.lastName | Last name. **Type:** String |
| Teacher.assignedSubjects | Subjects teacher is responsible for. **Type:** String |
| Teacher.markAttendance(studentId:int, status:enum) | Marks attendance for a student. |
| Teacher.enterMarks(studentId:int, subject:String, marksObtained:decimal) | Enters marks for a student. |

| | |
|---|---|
| Teacher.addBehaviorRemark(studentId:int, remarkText:String, remarkType:String) | Adds behavior remark. |
| Teacher.reviewComplaint(complaintId:int, status:enum) | Updates complaint status. |
| Teacher.postNotice(title:String, content:String) | Posts a notice. |
| User | Generic system user. **Attributes:** userId:int, username:String, email:String, password:String (encrypted), role:enum {Admin, Teacher, Student, Parent}, isActive:boolean |
| User.userId | Primary key. **Type:** int |
| User.username | Login username. **Type:** String |
| User.email | User email. **Type:** String |
| User.password | Encrypted password. **Type:** String |
| User.role | User role. **Type:** enum |
| User.isActive | Account status. **Type:** boolean |
| User.login(email:String, password:String) | Authenticates user. |
| User.logout() | Logs out user. |
| User.updateProfile(updatedData:Object) | Updates profile data. |
| UserRole | Maps users to roles. **Attributes:** userRoleId:int, userId:int (FK), roleId:int (FK) |
| UserRole.userRoleId | Primary key. **Type:** int |
| UserRole.userId | Foreign key to User. **Type:** int |
| UserRole.roleId | Foreign key to Role. **Type:** int |
| Role | Defines a system role. **Attributes:** roleId:int, roleName:String, description:String |
| Role.roleId | Primary key. **Type:** int |
| Role.roleName | Name of role. **Type:** String |

| | |
|---|---|
| Role.description | Role description. **Type:** String |

- **Structured Approach: Functions and Function Parameters:**

| **Function Name** | **Parameters** | **Description** |
|---|---|---|
| login | email:String, password:String | Authenticates a user using email and password. Returns success/failure and role-based access. |
| logout | userId:int | Logs out the specified user, ending their session. |
| updateProfile | userId:int, updatedData:Object | Updates a user's profile with the provided data. updatedData can contain username, email, password, etc. |
| markAttendance | studentId:int, date:Date, status:enum {Present, Absent, Late} | Marks attendance for a student on a given date. |
| updateAttendance | attendanceId:int, status:enum {Present, Absent, Late} | Updates an existing attendance record. |
| lockAttendance | attendanceId:int | Locks attendance for a particular date to prevent further edits. |
| addMarks | studentId:int, subject:String, exam:String, marksObtained:decimal, totalMarks:decimal | Adds a new marks entry for a student for a specific exam and subject. |
| updateMarks | marksId:int, marksObtained:decimal | Updates marks for an existing record. |
| submitComplaint | submittedByUserId:int, assignedToUserId:int, description:String, category:String | Creates a new complaint record in the system. |

| | | |
|---|---|---|
| updateComplaintStatus | complaintId:int, newStatus:enum {Submitted, UnderReview, Resolved, Rejected, Closed} | Updates the status of a complaint. |
| addComplaintRemarks | complaintId:int, remarks:String | Adds remarks/comments to an existing complaint. |
| compilePortfolio | studentId:int | Gathers student data (attendance, marks, achievements, behavior) to generate portfolio. |
| updatePortfolio | portfolioId:int, updatedData:Object | Updates an existing portfolio with new data. |
| exportPortfolioPDF | portfolioId:int | Generates a PDF document of a student's portfolio. |
| sendNotification | userId:int, content:String, type:String | Sends a notification to a user. |
| markNotificationRead | notificationId:int | Marks a notification as read. |
| recordAuditLog | userId:int, action:String, details:String | Records an action performed in the system for auditing purposes. |
| updateFeePayment | feeId:int, paidAmount:decimal | Updates the paid amount for a student fee record. |
| calculateFeeDue | feeId:int | Calculates the remaining due amount for a fee record. |

- **Object-Oriented (OO) Approach: Objects, Attributes, Methods, & Method Parameters:**

***Object: User***

**Attributes**

| Attribute | Type | Description |
|---|---|---|
| userId | int | Unique system identifier |
| username | String | Login username |
| email | String | User email |
| password | String | Encrypted password |
| role | enum {Admin, Teacher, Student, Parent} | User role |
| isActive | boolean | Account status |

**Methods**

| Method | Parameters | Description |
|---|---|---|
| login | email:String, password:String | Authenticates user and grants access |
| logout | — | Ends user session |
| updateProfile | updatedData:Object | Updates user's profile attributes |

***Object: Teacher***

**Attributes**

| Attribute | Type | Description |
|---|---|---|
| teacherId | int | Unique identifier |
| userId | int | FK → User |
| firstName | String | First name |

| | | |
|---|---|---|
| lastName | String | Last name |
| assignedSubjects | String | Subjects taught |

**Methods**

| Method | Parameters | Description |
|---|---|---|
| markAttendance | studentId:int, status:enum {Present, Absent, Late} | Marks attendance for a student |
| enterMarks | studentId:int, subject:String, marksObtained:decimal | Records marks for a student |
| addBehaviorRemark | studentId:int, remarkText:String, remarkType:String | Adds behavioral remark |
| reviewComplaint | complaintId:int, status:enum | Updates complaint status |
| postNotice | title:String, content:String | Posts notice to students/parents |

***Object: Admin***

**Attributes**

| Attribute | Type | Description |
|---|---|---|
| adminId | int | Unique identifier |
| userId | int | FK → User |
| firstName | String | First name |
| lastName | String | Last name |

**Methods**

| Method | Parameters | Description |
|---|---|---|
| createUser | userData:Object | Creates a new system user |
| generateReport | reportType:String | Generates attendance, marks, portfolio, or risk report |

| | | |
|---|---|---|
| updateSettings | settings:Object | Updates system settings |

*Object: Attendance*

**Attributes**

| Attribute | Type | Description |
|---|---|---|
| attendanceId | int | PK |
| studentId | int | FK → Student |
| teacherId | int | FK → Teacher |
| date | Date | Attendance date |
| status | enum {Present, Absent, Late} | Attendance status |

**Methods**

| Method | Parameters | Description |
|---|---|---|
| markAttendance | status:enum | Records attendance |
| updateAttendance | status:enum | Updates existing attendance |
| lockAttendance | — | Locks record |

*Object: Marks*

**Attributes**

| Attribute | Type | Description |
|---|---|---|
| marksId | int | PK |
| studentId | int | FK → Student |
| teacherId | int | FK → Teacher |
| subject | String | Subject name |
| exam | String | Exam name |
| marksObtained | decimal | Marks obtained |

| | | |
|---|---|---|
| totalMarks | decimal | Maximum marks |

**Methods**

| Method | Parameters | Description |
|---|---|---|
| addMarks | marks:decimal | Adds marks for a student |
| updateMarks | marks:decimal | Updates marks |

***Object: Complaint***

**Attributes**

| Attribute | Type | Description |
|---|---|---|
| complaintId | int | PK |
| submittedByUserId | int | FK → User |
| assignedToUserId | int | FK → User |
| description | String | Complaint description |
| status | enum {Submitted, UnderReview, Resolved, Rejected, Closed} | Current status |
| submittedDate | Date | Date of submission |
| closedDate | Date | Date of closure |

**Methods**

| Method | Parameters | Description |
|---|---|---|
| submitComplaint | description:String | Submit new complaint |
| updateStatus | newStatus:enum | Update complaint status |
| addRemarks | remarks:String | Add remarks to complaint |

*Object: StudentPortfolio*

**Attributes**

| Attribute | Type | Description |
|---|---|---|
| portfolioId | int | PK |
| studentId | int | FK → Student |
| generatedOn | Date | Generation date |
| lastUpdated | Date | Last update |

**Methods**

| Method | Parameters | Description |
|---|---|---|
| compileData | — | Compile portfolio data from attendance, marks, achievements, behavior |
| updatePortfolio | — | Update portfolio with new data |
| exportPDF | — | Export portfolio to PDF |

*Object: Achievement*

**Attributes**

| Attribute | Type | Description |
|---|---|---|
| achievementId | int | Unique identifier (PK) |
| studentId | int | FK → Student |
| title | String | Achievement title |
| category | String | Achievement category |
| date | Date | Date of achievement |

| | | |
|---|---|---|
| filePath | String | Path to supporting document/file |

**Methods**

| Method | Parameters | Description |
|---|---|---|
| addAchievement | achievementData:Object | Adds a new achievement record for a student |
| updateAchievement | achievementId:int, updatedData:Object | Updates an existing achievement |

***Object: BehaviorRemark***

**Attributes**

| Attribute | Type | Description |
|---|---|---|
| remarkId | int | Unique identifier (PK) |
| studentId | int | FK → Student |
| teacherId | int | FK → Teacher |
| remarkText | Text | Remark content |
| remarkType | String | Type of remark (e.g., Positive, Negative, Neutral) |
| date | Date | Date the remark was recorded |

**Methods**

| Method | Parameters | Description |
|---|---|---|
| addRemark | text:String, type:String | Adds a new behavioral remark for a student |
| updateRemark | text:String | Updates an existing remark |

*Object: Fee*

**Attributes**

| Attribute | Type | Description |
|---|---|---|
| feeId | int | Unique identifier (PK) |
| studentId | int | FK → Student |
| totalAmount | decimal | Total fee amount |
| paidAmount | decimal | Amount already paid |
| dueDate | Date | Payment due date |
| status | enum {Unpaid, Partial, Paid, Overdue} | Current fee status |

**Methods**

| Method | Parameters | Description |
|---|---|---|
| updatePayment | amount:decimal | Updates payment for the student fee |
| calculateDue | — | Calculates remaining amount due |

*Object: Message*

**Attributes**

| Attribute | Type | Description |
|---|---|---|
| messageId | int | Unique identifier (PK) |
| senderId | int | FK → User (sender) |
| receiverId | int | FK → User (receiver) |
| content | Text | Message content |

| | | |
|---|---|---|
| sentAt | DateTime | Timestamp when message is sent |
| status | String | Message status (e.g., Read, Unread) |

**Methods**

| Method | Parameters | Description |
|---|---|---|
| sendMessage | content:String, receiverId:int | Sends message to a user |
| readMessage | — | Marks message as read |

***Object: Notification***

**Attributes**

| Attribute | Type | Description |
|---|---|---|
| notificationId | int | Unique identifier (PK) |
| userId | int | FK → User (recipient) |
| content | Text | Notification content |
| dateSent | Date | Date notification was sent |
| status | String | Status (e.g., Unread, Read) |

**Methods**

| Method | Parameters | Description |
|---|---|---|
| sendNotification | content:String | Sends notification to a user |
| markRead | — | Marks notification as read |

***Object: AuditLog***

**Attributes**

| Attribute | Type | Description |
|---|---|---|

| | | |
|---|---|---|
| logId | int | Unique identifier (PK) |
| userId | int | FK → User (who performed action) |
| action | String | Action description |
| timestamp | Timestamp | Time action was performed |
| details | Text | Additional details about the action |

**Methods**

| Method | Parameters | Description |
|---|---|---|
| recordAction | userId:int, action:String, details:String | Records an administrative or system action |

## 4.7 User Interface Design

The system provides a **role-based interface**, tailored to each type of user: Admin, Teacher, Student, and Parent. After logging in, the user sees a **dashboard** that gives access only to features relevant to their role.

- **Admin:**
    a. Can manage all users (students, teachers, parents)
    b. Add/edit/delete records for fees, subjects, and classes
    c. Generate reports (attendance, marks, portfolio, risk monitoring)
    d. Review complaints and notifications
    e. Audit logs to monitor system activity
- **Teacher:**
    a. Record student attendance
    b. Enter marks for exams and assignments
    c. Add behavioral remarks
    d. Upload student achievements
    e. Review and resolve complaints
    f. Post notices to students/parents
- **Student:**
    a. View attendance, marks, and achievements
    b. Access personal portfolio
    c. Submit complaints
    d. Receive notifications from teachers/admins

- **Parent:**
    a. View their child's academic records (attendance, marks, portfolio)
    b. Communicate with teachers via messages
    c. Submit complaints regarding their child

**User Feedback:**

- The system provides **real-time feedback** for user actions:
    a. Success messages (e.g., "Attendance successfully recorded")
    b. Error messages (e.g., "Marks cannot exceed total marks")
    c. Validation messages for input fields
    d. Notifications for important events (new message, new complaint, fee due)

The interface is designed for **clarity, responsiveness, and ease of navigation**, ensuring users can complete tasks efficiently.

## 1. Screen Images

Login Screen

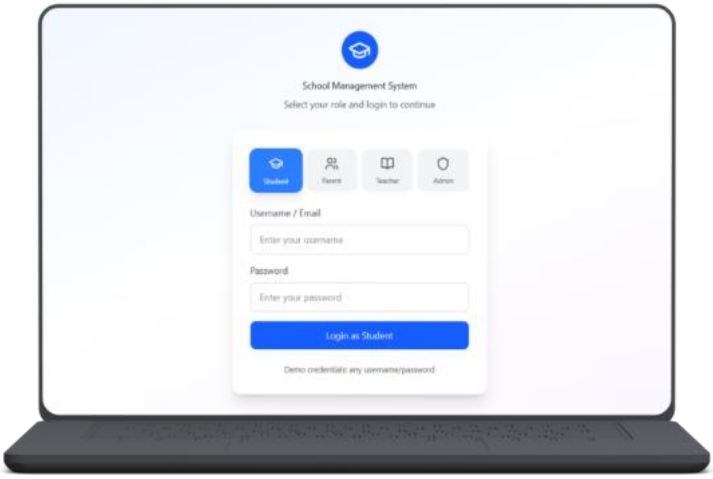



*Figure 13 Screen Image 1- Login Screen*

Student Portfolio Screen

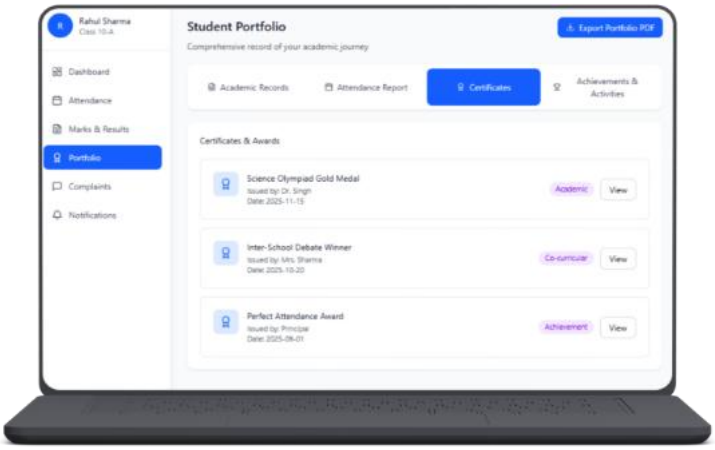



*Figure 14 Screen Image 2- Student Portfolio*

Attendance Management Screen(Teacher)

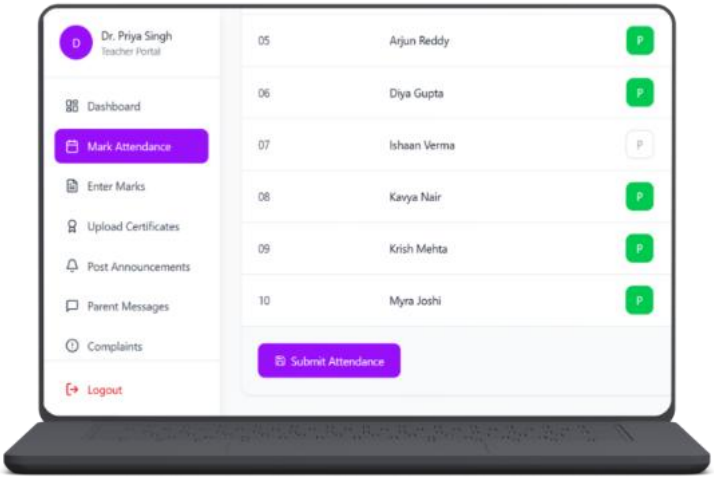



*Figure 15 Screen Image 3- Attendance Management*

Complaint Feedback Screen (Student)

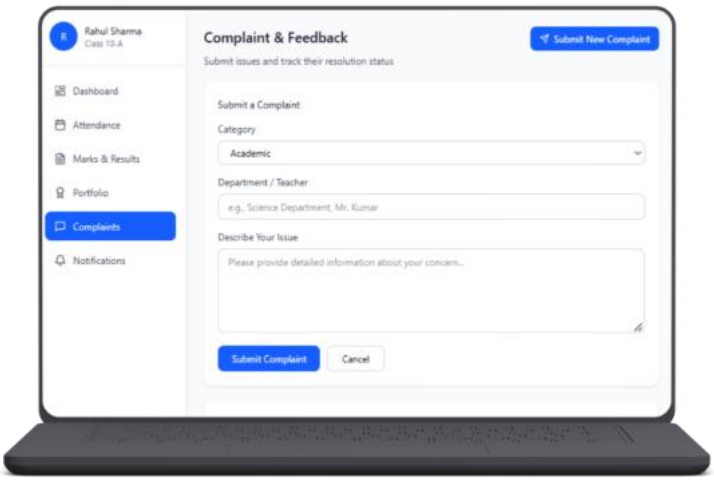



*Figure 16 Screen Image 4- Complain Feedback*

Student Portfolio Generation Screen

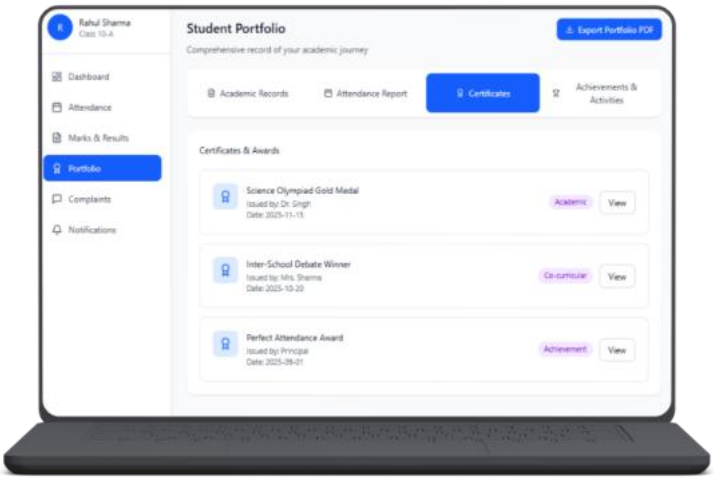



*Figure 17 Screen Image 5- Student Portfolio Generation*

## 2. Screen Objects and Actions

### *Use Case 1: Submit Complaint*

- **Screen Objects:**
  a. Complaint Text Area → where user types the complaint
  b. Submit Button → submits complaint to system
  c. Status Message Label → shows success/error
- **Actions:**
  a. User enters complaint text
  b. User clicks Submit

c. System validates input
d. System saves complaint
e. Status message shows confirmation

*Use Case 2: Mark Attendance*

- **Screen Objects:**
  a. Student List Table → shows students' names
  b. Attendance Controls → dropdown or radio buttons: Present / Absent / Late
  c. Submit Button → saves attendance
  d. Message Area → shows success/error
- **Actions:**
  a. Teacher selects status for each student
  b. Teacher clicks Submit
  c. System validates input
  d. System updates attendance records
  e. Success message appears

## 4.8 Design Decisions

This section describes the major design decisions taken during the development of the **School Management System (SMS)**. Each decision is aligned with the requirements specified in the SRS and justified based on system efficiency, maintainability, scalability, and security.

### 1. Design Paradigm

**Decision:** Object-Oriented Design (OOD)

**Description:**
The system is designed using an object-oriented approach where real-world entities such as User, Student, Teacher, Parent, Attendance, Marks, and Fee are modeled as objects with attributes and behaviors.

**Justification:**

- Improves modularity and reusability
- Simplifies maintenance and future enhancement
- Supports UML-based modeling (Class and Sequence Diagrams)

### 2. Architectural Style

**Decision:** Layered (Multi-Tier) Architecture

**Description:**
The system follows a layered architecture consisting of Presentation, Application, Data Access, and Database layers.

**Justification:**

- Ensures separation of concerns
- Allows independent modification of layers
- Enhances scalability and testability

## 3. User and Role Management

**Decision:** Role-Based Access Control (RBAC)

**Description:**
The system supports four predefined roles: Admin, Teacher, Student, and Parent, as defined in the SRS.

**Justification:**

- Ensures secure and controlled access
- Prevents unauthorized operations
- Simplifies permission management

## 4. Database Design Approach

**Decision:** Relational Database Design

**Description:**
A relational database is used to store persistent system data.

**Justification:**

- Ensures structured data storage
- Maintains referential integrity
- Supports transactional consistency

## 5. Normalization Level

**Decision:** Third Normal Form (3NF)

**Description:**
The database schema is normalized up to Third Normal Form.

**Justification:**

- Reduces redundancy
- Prevents update anomalies
- Improves data consistency

## 6. Object–Relational Mapping Strategy

**Decision:** Logical Mapping between Classes and Tables

**Description:**
Each UML class corresponds to a relational database table.

**Justification:**

- Maintains consistency between design and implementation
- Simplifies CRUD operations
- Improves traceability between ER and Class Diagrams

## 7. Algorithmic Approach

**Decision:** Rule-Based and Transaction-Oriented Logic

**Description:**
Business processes such as attendance management, fee calculation, and complaint handling use rule-based logic.

**Justification:**

- Avoids unnecessary computational complexity
- Enhances readability and maintainability
- Suitable for academic management systems

## 8. Behavioral Modeling Strategy

**Decision:** Use-Case Driven Sequence Modeling

**Description:**
Sequence diagrams are derived from major SRS use cases, particularly **User Login & Role-Based Access**.

**Justification:**

- Login is the primary system entry point
- Demonstrates interaction between multiple system components
- Ensures consistency with SRS workflows

## 9. Security Design

**Decision:** Application-Level Security Enforcement

**Description:**
Security is enforced through encrypted password storage, role validation, and audit logging.

**Justification:**

- Protects sensitive academic and personal data
- Ensures accountability
- Prevents unauthorized access

## 10. Modularity and Component Design

**Decision:** Component-Based Modular Design

**Description:**
The system is divided into logical components such as User Management, Academic Records, Complaints & Fees, and System Services.

**Justification:**

- Improves system organization
- Enhances maintainability and scalability
- Aligns with UML Component Diagram

## 11. Traceability and Documentation

**Decision:** Strict SRS-to-SDS Traceability

**Description:**
All design artifacts are derived directly from the SRS.

**Justification:**

- Ensures requirement coverage
- Prevents scope creep
- Supports academic validation

## 12. Technology Stack Selection

**Decision:** ASP.NET Core, React.js, and SQL Server

**Description:**
The backend of the system is implemented using **ASP.NET Core Web API**, the frontend using **React.js**, and **SQL Server** is used as the relational database.

**Justification:**

- Supports scalable web-based applications
- Enables secure and high-performance backend services
- Aligns with industry standards and university curriculum

- Facilitates maintainable and modular development

### 13. Communication Strategy

**Decision:** RESTful API Communication (JSON over HTTP)

**Description:**
The system follows a client–server model where the frontend communicates with the backend through RESTful APIs using JSON.

**Justification:**

- Decouples frontend and backend components
- Enables stateless communication
- Simplifies testing and system integration

### 14. Error Handling and Validation Strategy

**Decision:** Server-Side and Client-Side Validation

**Description:**
Input validation and error handling are implemented at both client and server levels.

**Justification:**

- Client-side validation improves user experience
- Server-side validation ensures data integrity and security
- Prevents invalid or malicious data submission

## 4.9 Summary

The Software Design Specification (SDS) for the School Management System defines the architectural design, component structure, data organization, and interaction mechanisms of the proposed system. The design adopts a modular and layered approach to ensure scalability, maintainability, and role-based access control. Core system functionalities are divided into user management, academic records, finance and complaints handling, and system services such as notifications and audit logging. UML diagrams, including class and sequence diagrams, are used to clearly represent static structures and dynamic behavior of the system. The SDS provides a strong foundation for implementation by ensuring consistency with system requirements, promoting code reusability, and supporting future system enhancements.

# References

# References

## 1. Article in a Journal

[1] W. Ahmed, Q. Parveen, and M. A. Dahar, "Role of School Management Systems in Improving Administrative Efficiency in Pakistan," *South Asian Journal of Educational Research*, vol. 5, no. 2, pp. 45–58, 2022.

[2] H. Muslim, N. Aslam, and T. Rizwan, "Enhancing Parent–Teacher Communication through Digital Platforms in Pakistani Schools," *Journal of Social Sciences Research Pakistan*, vol. 11, no. 2, pp. 32–49, 2025.

[3] M. Urooj, A. Tariq, and S. Khan, "Student Portfolio Management and Digital Record-Keeping in School Environments," *Pakistan Journal of Educational Technology*, vol. 10, no. 1, pp. 60–75, 2025.

[4] R. Khan, T. Ali, and S. Qazi, "Centralized Digital Noticeboards and Notification Systems in Schools: Improving Communication Efficiency," *Asian Journal of Educational Management*, vol. 15, no. 3, pp. 210–225, 2024.

[5] S. Ahmed, A. Farooq, and K. Shah, "Behavior and Complaint Management in School Management Systems: A Structured Approach," *International Journal of School Administration*, vol. 8, no. 2, pp. 101–118, 2023.

## 2. Articles from Conference Proceedings (published)

[6] F. Malik and M. Hussain, "Integrating Student Portfolios with Parent–Teacher Communication Hubs for Pakistani Schools," in *Proceedings of the International Conference on Educational Technologies*, Lahore, Pakistan, 2024, pp. 45–52.

[7] N. Rehman and H. Ali, "Digital Transformation in School Administration: Lessons from South Asia," in *Proceedings of the 5th International Symposium on Educational Management Systems*, Islamabad, Pakistan, 2023, pp. 60–68.

## 3. World Wide Web

[8] EduSuite, “Campus Management Software – EduSuite (Pakistan),” *EduSuite*. [Online]. Available: https://www.edusuite.pk/campus-management-system. [Accessed: Nov. 9, 2025].

[9] GIFT University LMS, “GIFT University Learning Management System,” *GIFT University LMS*. [Online]. Available: https://lms.gift.edu.pk. [Accessed: Nov. 9, 2025].

[10] FAST National University LMS, “Learning Management System – FAST NUCES,” *FAST NUCES LMS*. [Online]. Available: https://lms.nu.edu.pk. [Accessed: Nov. 9, 2025].

[11] University of Management and Technology (UMT), “UMT LMS Portal,” *UMT LMS*. [Online]. Available: https://lms.umt.edu.pk. [Accessed: Nov. 9, 2025].

[12] IISAT, “IISAT Learning Management System,” *IISAT LMS*. [Online]. Available: https://lms.iisat.edu.pk. [Accessed: Nov. 9, 2025].

## 4. ChatGPT

[13] OpenAI, ChatGPT, "Discussion on Final Year Project Proposal, SRS, SDS, and Prototype Preparation for School Management System." Accessed on: Feb. 1, 2026. [Online]. Available: https://chat.openai.com/

[14] OpenAI, ChatGPT, "Guidance on Front-End React & Tailwind Integration for School Management System Prototype." Accessed on: Feb. 1, 2026. [Online]. Available: https://chat.openai.com/

[15] OpenAI, ChatGPT, "Assistance with Literature Review, Gap Analysis, and IEEE Referencing for FYDP." Accessed on: Feb. 1, 2026. [Online]. Available: https://chat.openai.com/

[16] OpenAI, ChatGPT, "Design Specification & Requirement Mapping with Scope Table for School Management System." Accessed on: Feb. 1, 2026. [Online]. Available: https://chat.openai.com/

[17] OpenAI, ChatGPT, "Success Criteria, Risks, Assumptions, and Project Roles for FYDP Documentation." Accessed on: Feb. 1, 2026. [Online]. Available: https://chat.openai.com/